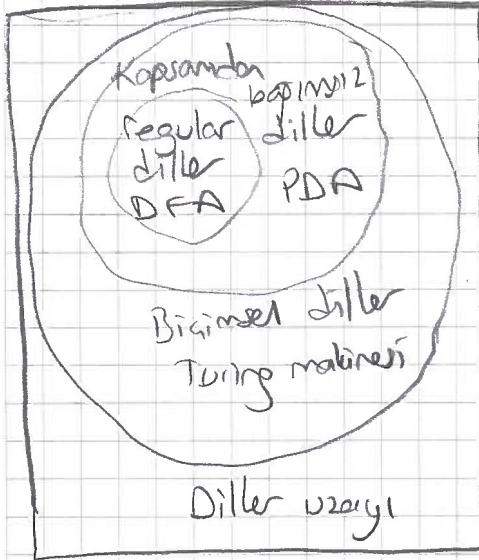


01.10.2015 (Computing Theory)
Bicimsel Diller Neuzat Tarhan

- Regular diller
- Deterministic Finite Otomata (DFA)
- Kapsamda bapımız diller
- PDA



Bicimsel dilleri tanıyan makineler

- ↳ DFA
- ↳ PDA (Push-Down Automata)
- ↳ Turing makinesi

Bicimsel olmayan diller, (örneğin) ütü konular kapsamındadır.

Ünöl Yarımaçan
Bicimsel Diller ve Özdeşlik Kuramı
-Bicimler Yayıncısı

Ara sınav	% 30	} → yıl içi %50
Kısa sınav 1	% 10	
Kısa sınav 2	% 10	
Ödev	% 10	
		Final %50

* Regular ifadeler, DFA anlatılacak. DFA'nın anlatıldığı hafta kısa sınav yapılacaktır. Başit bir DFA tanımlaması tarzında soru olacak.

* İkinci kısa sınavın notu devamdan gelecek.

* Sınavda "regular ifade" ↔ DFA denklemleri çıkıyor.

* Üze → Düzenli olmayan ifadeler son konu.

DFA: Geçici bellek yok. Anlık gelen işi yapar ^{sadece}.
 Push Down Automata: Yığıt bellek var.
 Turing makinesi: Random Access Memory.

① Alfabeye (Σ): Dil tanımlamak için kullanılan sonlu sayıdaki simge kümesine denir.

Baş karakter: $\lambda, A, E, \wedge$

Bitleştirme: $wv, w.v, w \circ v$
 (Concatenation)

} Hepsisi kullanılabilir.

Üs işlemi: Karakterin üs işlemi, üs kadar bitleştirilmiştir.

$$(abba)^2 = (abba)(abba) = abbaabba$$

$$w^0 = \lambda$$

Klosure işlemi (yıldız kapama) $\Sigma^* = \bigcup_{i=0}^{\infty} \Sigma^i$

Σ^* : Σ üzerinde tanımlı olan bütün katarlar kümesi

Bu işlem sonucunda ilgili alfabenin evrensel kümesi çıkar.
 Bunun bir alt kümesi, bizim dilimizdir.

+ işlemi (pozitif kapama) Yıldız kapamadan λ çıkarıldıktan sonra ortaya çıkan kümedir.

Dilde bitleştirme: Dillerin elemanlarının kartezyen çarpımıdır. aynı olanlar çıkarılır.

Dilde üs alma: Aynı katar gibidir. Kendisiyle bitleştirilir.

$$L^0 = \{\lambda\}$$

$$L^* = \bigcup_{i=0}^{\infty} L^i$$

DFA - regüler ifadeyi tanıır (sadece)
regüler ifadeyi sadece DFA tanıır (???)

Regüler ifade

R , bir regex ise;

$L(R) \Rightarrow R$ 'nin tanımladığı dil.

Örnek: $L(abc) = \{abc\}$

$L(\text{evet}|\text{hayır}) = \{\text{evet}|\text{hayır}\}$

$L(1(0|1)^*) = 1$ ile başlayan tüm ikili sayılar

Özellikler:

- 1) $*$
- 2) birleştirme
- 3) $|$ (veya)

Örnek:

$ab^*|c$

- $ac \rightarrow \text{olmaz}$
- $c \rightarrow \text{olur}$

15.10.2015

Bianmel Dille

①

Örnek:

En fazla 3 terimden oluşan regüler dil

Cevap: $(a|b|x)(a|b|\lambda)(a|b|\lambda)$ şeklinde olabilir.

Örnek:

İçinde tek sayıda sembol bulunan ve sonu "a" ile biten katarlardan oluşan dili yazın.

Cevap: $(aa|ab|ba|bb)^*a$

Örnek: İçinde çift sayıda sembol bulunan tüm katarlar:

Cevap: $(aa|ab|ba|bb)^*$

Örnek: $\Sigma = \{0,1,2,\dots,9\}$

rakam = $\{0,1,2,\dots,9\}$

D = $\{1,2,3,\dots,9\}$

sayılar = $D(rakam)^*$

tamsayı = $(-|E)sayılar|0$

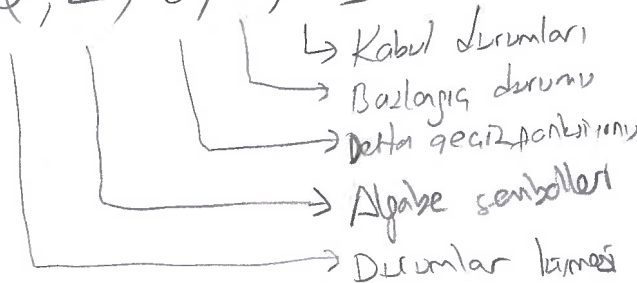
reel sayı = tamsayı $(E|(.sayılar))$

* Leksema: Programlama dilinde en küçük anlamlı birim.

* Pascal Unsigned int değeri pozitif.

DFA

DFA $M = \{Q, \Sigma, \delta, S, F\}$



① Durum

→ ○ Başlangıç durumu

⊙ Kabul durumu

① $\xrightarrow{a}$ ① Geçiş fonksiyonu

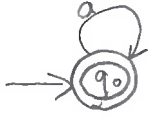
$(q, a) \rightarrow q'$

15.10.2015

Diğer Diller

2

Örnek: $\Sigma = \{a\}$ (alfabe $\{a\}$ dan) DFA'yı çiz

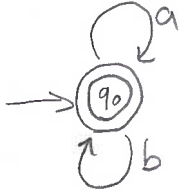


Örnek: $\Rightarrow a^*b$

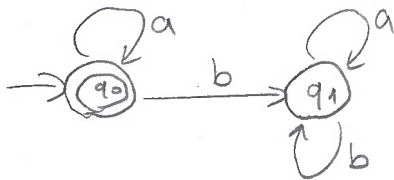
* Dilde boş katar kabul ise başlangıç durumu kabuldur.



Örnek: $\Sigma = \{a,b\}$ Dil = $(a|b)^*$ DFA çiz:



Örnek: $\Sigma = \{a,b\}$ Dil = a^* DFA çiz.



* Q'daki durum sayısı $\times$ Alfabe'deki sembol sayısı

Geçer sayısını verir.

Örnek: $\Sigma = \{a\}$, içinde çift sayıda katarları kabul eden dili
fazan ve matematiksel ifadesi yazınız.

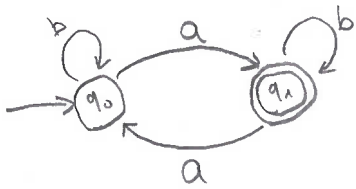
Cevap: $L = \{a^{2n} : n \geq 0\}$ Regüler: $(aa)^*$

Açık gösterim: $L = \{ \lambda, aa, aaaa, \dots \}$



Örnek: $\Sigma = \{a, b\}$ ve tamde tek sayıda a dan makine/

Cevap $n=0 \Rightarrow a^1$ dir. Başlangıç kabul değil. $\Sigma^* = \{ \lambda, a, b, aa, ab, ba, bba, \dots \}$
 $\times \quad \checkmark \quad \times \quad \times \quad \checkmark \quad \checkmark \quad \times \quad \dots$



DFA $M = \{Q, \Sigma, \delta, S, F\}$

$Q = \{q_0, q_1\} \rightarrow \delta = \underbrace{Q \times \Sigma \rightarrow Q}$
 $\Sigma = \{a, b\} \rightarrow \delta = \{(q_0, a), (q_0, b), (q_1, a), (q_1, b)\}$

$S = q_0$

$F = \{q_1\}$

Örnek: İstediği makine için $(w_1 = abba)$ kabul edilir, $(w_2 = aaba)$ red eder.

$\delta^*(q_0, abba) \rightarrow (q_1, bba) \rightarrow (q_1, ba) \rightarrow (q_1, a) \rightarrow (q_0) \notin F$ red

$\delta^*(q_0, aaba) \rightarrow (q_1, aba) \rightarrow (q_0, ba) \rightarrow (q_0, a) \rightarrow (q_1) \in F$ kabul

Soru: Trajik 1291 $\begin{pmatrix} K \\ S \\ T \end{pmatrix}$

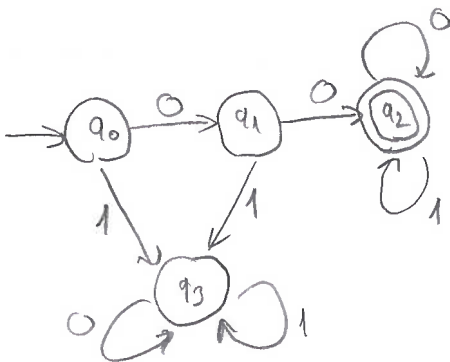
K	-	-
-	S	-
-	4	1

 1. durum 2. durum 3. durum

Bunun makinesini çiz

Soru: $(4k+1)$ aocuk ile oyun oynayacağız sayılara göre DFA

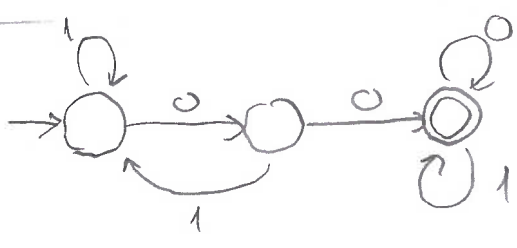
Örnek: $\Sigma = \{0, 1\}$ $L = 0a(0|1)^*$



$\Sigma^* = \{ \lambda, 0, 1, 00, 01, 10, 11, 001, \dots \}$

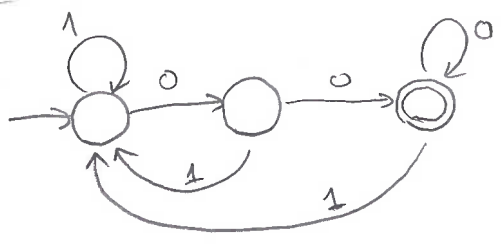
$0(0|1)^*0$

Überr: $\Sigma = \{0,1\}$ $L = (1|0)^* 00 (0|1)^*$



3 durum var.
2 sembol var
 $3 \times 2 = 6$ geçiş var

Überr: $\Sigma = \{0,1\}$ $L = (0|1)^* 00$

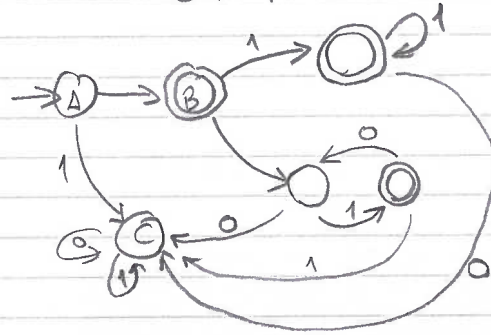


3 durum, 2 sembol
 $3 \times 2 = 6$ geçiş

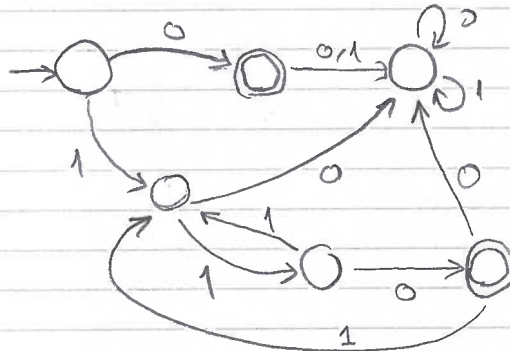
Überr: $01^* | 0(01)^*$

Örnek:

$$01^* \mid 0(01)^*$$

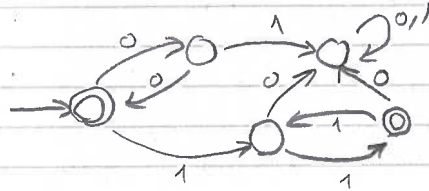


6 durum, 2 sembol
 $6 \times 2 = 12$ geçiş.

Örnek: $(11 \mid 110)^* 0$ DFA çizelim

$6 \times 2 = 12$ geçiş.

Ters Örnek:

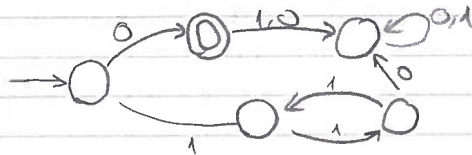


İfadeyi yazalım.

$\lambda \Rightarrow$ kabul olmalı
 İlk sol bölüme göre $(00)^*$ var bir yerde.
 Sağ alttan dolayı $(11)^+$ var.
 Sağa doğru: $(00)^* (00)^* (11)^+$

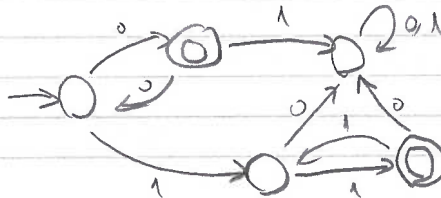
Veya daha sade: $(00)^* (11)^*$

Örnek:



$\Rightarrow (11)^+ \mid 0$

Örnek:



$\Rightarrow 0(00)^* \mid (00)^* (11)^+$

L1
DFA1

DFA Kurulum ve birleşimi ve farkı
 $\Sigma = \{a, b\}$ alfabesini kullanan DFA'lar olsun.

Taahhütte ağıt sağıda a olan DFA



makine = DFA $M_1 = \{Q_1, \Sigma_1, \delta_1, S_1, f_1\}$

$Q_1 = \{q_0, q_1\}$

$\Sigma = \{a, b\}$

$S_1 = q_0$

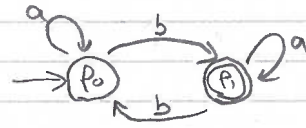
$f_1 = \{q_0\}$

$\delta_1 = \begin{cases} (q_0, a) \rightarrow q_1 \\ (q_0, b) \rightarrow q_0 \\ (q_1, a) \rightarrow q_0 \\ (q_1, b) \rightarrow q_1 \end{cases}$

L2
DFA2

Taahhütte tek sağıda b olan katarları kabuleden makine

DFA $M_2 = \{Q_2, \Sigma_2, \delta_2, S_2, f_2\}$



$Q_2 = \{p_0, p_1\}$

$\Sigma_2 = \{a, b\}$

$S_2 = p_0$

$f_2 = \{p_1\}$

$\delta_2 = \begin{cases} (p_0, a) \rightarrow p_1 \\ (p_0, b) \rightarrow p_0 \\ (p_1, a) \rightarrow p_0 \\ (p_1, b) \rightarrow p_1 \end{cases}$

İki makineyi birleştireceğiz

Yeni makine için $Q = Q_1 \times Q_2$ (kardesim arşim)
 Örnek için $Q = \{q_0p_0, q_0p_1, q_1p_0, q_1p_1\}$
 $\Sigma = \{a, b\}$
 $S = S_1S_2 \Rightarrow S = q_0p_0$

$\delta = \begin{cases} (q_0p_0, a) \rightarrow q_1p_0 \\ (q_0p_0, b) \rightarrow \\ (q_0p_1, a) \rightarrow \\ (q_0p_1, b) \rightarrow \\ (q_1p_0, a) \rightarrow \\ (q_1p_0, b) \rightarrow \\ (q_1p_1, a) \rightarrow \\ (q_1p_1, b) \rightarrow \end{cases}$

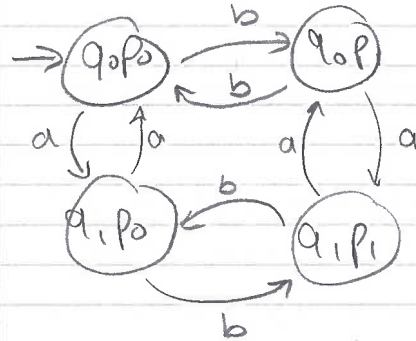
Yukarıda
gösterildiği
↑

$(q_0, a) \Rightarrow q_1$
 $(p_0, a) \Rightarrow p_0$

Türkeri
Yonker

Tablo

Q	a	b
q_0p_0	q_1p_0	q_0p_1
q_0p_1	q_1p_1	q_0p_0
q_1p_0	q_0p_0	q_1p_1
q_1p_1	q_0p_1	q_1p_0



Şimdi kabul durumlarına bakalım.

Kesim = $f_1 = f_1 \times f_2 = q_0p_1$ (içinde çift sayıda a VE tek sayıda b den)Birleşim = $f_u = f_1 \times Q_2 \cup Q_1 \times f_2$ (çift sayıda a VE YA tek sayıda b)

Aynı kabul durumu ile 2.

$$= \{q_0p_0, q_0p_1\} \cup \{q_0p_1, q_1p_1\}$$

$$= \{q_0p_0, q_0p_1, q_1p_1\} \rightarrow \text{kesimlerin}^{\text{billeri}} \text{ çıkarıldı.}$$

Fark = $f_{u-u} = f_1 \times Q_2 - Q_1 \times f_2$ (çift sayıda a olsun, tek b olmasın)

$$= \{q_0p_0, q_0p_1\} - \{q_0p_1, q_1p_1\}$$

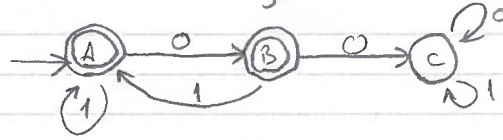
$$= \{q_0p_0\}$$

Örnek:

Ticründe (00) alt katarını ticrümeyen (ve veya fark) sonu (10) ile biten

Gözetim: $\Sigma = \{0,1\}$ dır.

Önce 1. DFA'yı çizelim.

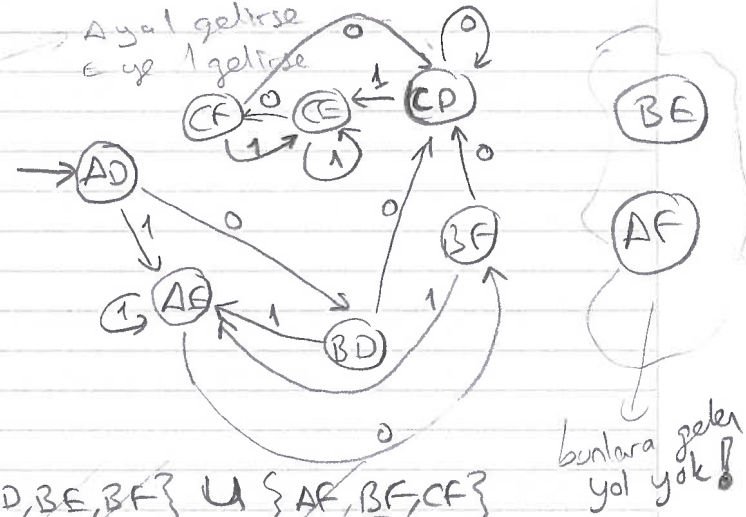


İkinci DFA'yı çizelim.



Bir araya
getirelim
Başlangıç
Gökten
kullanılmamış

	0	1
AD	BD	AE
AE	BF	AE
AF	BD	AE
BD	CD	AE
BE	CF	AE
BF	CD	AE
CD	CD	CE
CE	CF	CE
CF	CD	CE



Birleştirim: $F_u = \{AD, AE, AF, BD, BE, BF\} \cup \{AF, BF, CF\}$
 $= \{AD, AE, BD, BF, CF\}$

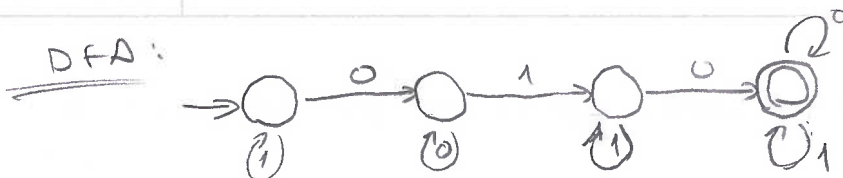
Kesilim: $F_n = F_1 \times F_2 = \{AF, BF\} \Rightarrow \{BF\}$

Fark: $F_{u-2} = \{AD, AE, BD\}$

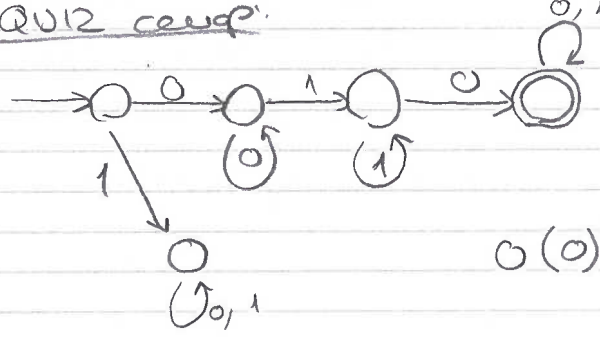
Q12

$\Sigma = \{0,1\}$ 0 ile başlayan ve en az 1 tane 1 katarı
 içeren regüler ifade ve DFA makinesi

ifade: $0(0/1)^*01(0/1)^*$



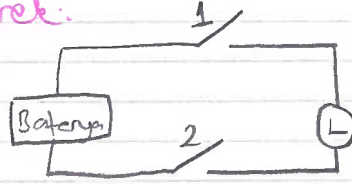
QUIZ cevap:



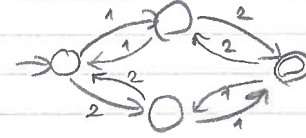
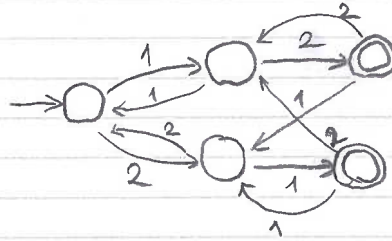
$$0(0)^*1(1)^*0(1|0)^*$$

Nerajit haccin potansiyel srau sorusu

örnek:



Basta her iki anahtar aek. Laaba yanma durumunu çizelim. Anahtarlar her basısta on-off durum deđitiriyor.



Potansiyel srau sorusu.

örnek: Elektrikli batterjiye var. Döğmesi var. Yukarı ve aşağı hareket ediyor. Bunun cakuma durumunun DFA'sını çiziniz. Batarya durumu 0 olsun.



0: Kapatı
1: Açık
2: Gidesiçak

A: aşağı
Y: yukarı

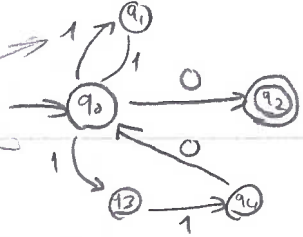


NFA - Nondeterministic Finite Automata

$\delta(q, \Lambda) = q \Rightarrow$ bir katarın yanına bue katar gelirse kendiridir.

Alfabedeki her sembol için aıkuk gösterilmeyebilir, birden fazla da gösterilebilir. Burada bizi sadece kabul durumu ilgilendiriyor. Red durumları bizi ilgilendirmez.

NFA'nın DFA'den farkları.

örnek: $(11+1110)^*0$ NFA $M = \{Q, \Sigma, \delta, s, F\}$ $Q = \{q_0, q_1, q_2, q_3, q_4\}$ $\Sigma = \{0, 1\}$ $s = \{q_0\}$ $F = \{q_2\}$ $\delta \Rightarrow$ en az $Q \times \Sigma$, en fazla 2^Q

* 1'egiden var

* Red durumları yok.

Devam: Geçişleri yazalım.

$$\begin{aligned} \delta &= (q_0, 0) \rightarrow q_2 \\ (q_0, 1) &\rightarrow \{q_1, q_2\} \\ (q_1, 1) &\rightarrow q_0 \\ (q_3, 1) &\rightarrow q_4 \\ (q_4, 0) &\rightarrow q_0 \end{aligned} \text{ şeklinde yazılır.}$$

** Şimdi bu makineyi çalıştırdım. w , bir katedir. $w = ya$ olsun. $y \in \Sigma^*$ ve $a \in \Sigma$
kater bir sembol

$\delta^*(s, w)$ olursa ne olur, buna bakalım.

$$\delta^*(q, w) = \delta(q, ya) = \bigcup_{r \in \delta(q, y)} r$$

** Aynı makine için, $w = 111$, $s = q_0$ durumuna bakalım

$$\delta^*(q_0, 111) \rightarrow (\{q_1, q_3\}, 11) \rightarrow (\{q_0, q_4\}, 1) = \{q_1, q_3\}$$

Ne zaman kabul $\Rightarrow \delta^*(s, w) \cap F \neq \emptyset$ ise kabuldür.

Örneğin 11110 kabul mü?

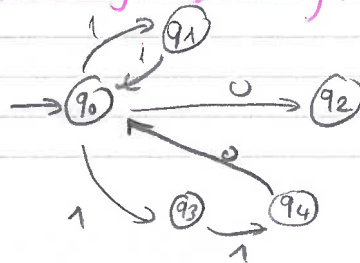
$$\begin{aligned} \delta^*(s, w) &= \delta^*(q_0, 11110) \\ &\rightarrow (\{q_1, q_3\}, 1110) \\ &\rightarrow (\{q_0, q_4\}, 110) \\ &\rightarrow (\{q_1, q_3\}, 10) \\ &\rightarrow (\{q_0, q_4\}, 0) \\ &\rightarrow (\{q_2, q_0\}) \end{aligned}$$

$$\delta^*(s, w) \cap F \text{ 'ye bakalım: } (\{q_2, q_0\}) \cap \{q_2\}$$

$\neq \emptyset$ olduğundan kabuldür.

NFA'yı DFA'ya çevirme

Başlangıçtan başlanır. Alfabenin her bir sembolü için duruma bakılır.



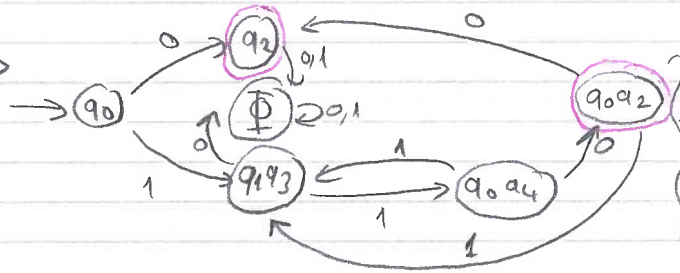
$q_0 q_2 \leftrightarrow q_2 q_0$
AYNI

$\delta(q_0, 0) = \{q_2\}$ yeni
 $\delta(q_0, 1) = \{q_1, q_3\}$ yeni
 $\delta(q_2, 0) = \emptyset$
 $\delta(q_2, 1) = \emptyset$
 $\delta(\emptyset, 0) = \emptyset$
 $\delta(\emptyset, 1) = \emptyset$
 $\delta(\{q_1, q_3\}, 0) = \emptyset$
 $\delta(\{q_1, q_3\}, 1) = \{q_0, q_4\}$ yeni
 $\delta(\{q_0, q_4\}, 0) = \{q_0, q_2\}$ yeni
 $\delta(\{q_0, q_4\}, 1) = \{q_1, q_3\}$ ✓
 $\delta(\{q_0, q_2\}, 0) = \{q_2\}$ ✓
 $\delta(\{q_0, q_2\}, 1) = \{q_1, q_3\}$ ✓

Aynı sonuç çıkar.

$\emptyset$	0	1
q_0	q_2	q_1, q_3
q_2	$\emptyset$	$\emptyset$
q_1, q_3	$\emptyset$	q_0, q_4
$\emptyset$	$\emptyset$	$\emptyset$
q_0, q_4	q_0, q_2	q_1, q_3
q_0, q_2	q_2	q_1, q_3

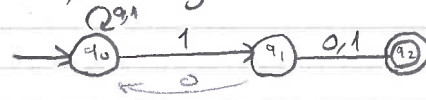
DFA'sını
çizelim



İçerisinde q_2 olan tüm durumlar kabuldür.

Örnek
Hocanın önerdiği

Sondan bir önceki semboldü 1 olan NFA'yı
çizip, DFA'ya çevirin.



101 katarı mı
bakalım

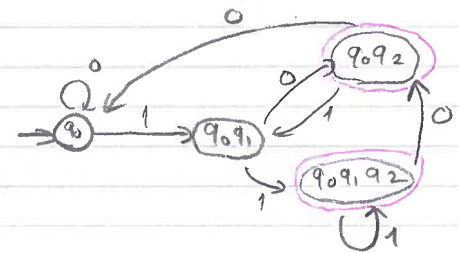
$\delta^*(s, w)$

$\delta^*(q_0, 101) \rightarrow (\{q_0, q_1\}, 01) \rightarrow (\{q_0, q_2\}, 1) \rightarrow \{q_1\}$

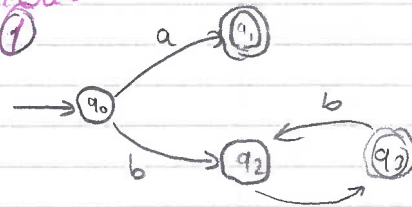
çözüm kısmı ile kısmını
bir küme oldu-
ğundan red olur.

Q	0	1
q_0	q_0	q_0, q_1
q_0, q_1	q_0, q_2	q_0, q_1, q_2
q_0, q_2	q_0	q_0, q_1
q_0, q_1, q_2	q_0, q_2	q_0, q_1, q_2

DFA'sı



Örnek 1



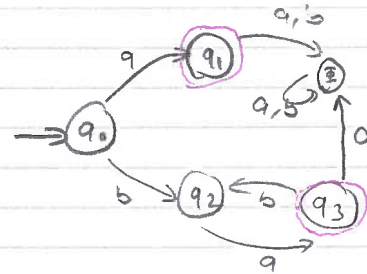
Regüler ifade nedir?

Cevap: $a|(ba)^+$ ya da $a|ba(ba)^*$

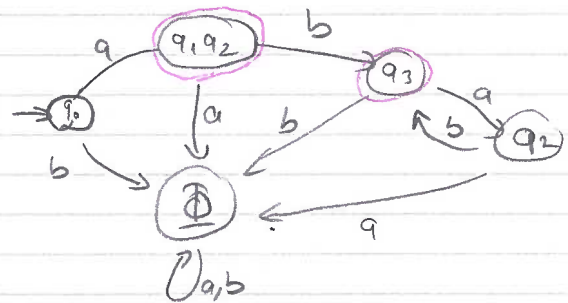
Örnek 2

1. makinenin DFA'sı $\Rightarrow$

Q	a	b
q0	q1	q2
q1	$\emptyset$	$\emptyset$
q2	q3	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$
q3	$\emptyset$	q1

2. makinenin DFA'sı $\Rightarrow$

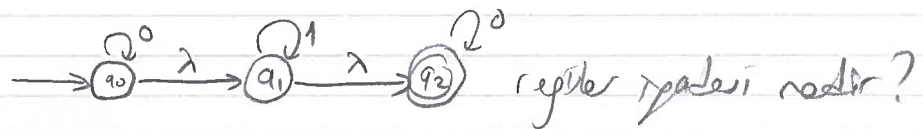
Q	a	b
q0	q1, q2	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$
q1, q2	$\emptyset$	q3
q3	q2	$\emptyset$
q2	$\emptyset$	q1



Baskül Geçerli NFA

$$\delta(q, \Lambda) = \{q'_1, q'_2, \dots, q'_n\}$$

Örnek:



regüler ifade nedir?

 $0^*1^*0^*$

00 kabul

 λ kabul

001 "

010 "

101 red

$$\delta^*(q, \Lambda) = \Lambda(q)$$

↪ boşluk kapama denir buna.
boşluk geçişi ile gidilebilecek tüm durumları gösterir.

Boşluk kapamayı bulmak önemli. Çünkü, kuantum programı gibi bir durumda iken diğer durumlarda da olma ihtimali vardır.

$$\delta^*(q, a) = \Lambda(\delta(\Lambda(q), a))$$

Boşluk kapamaları
Tüm durumları

$$\Lambda(q_0) = \{q_0, q_1, q_2\}$$

$$\Lambda(q_1) = \{q_1, q_2\}$$

$$\Lambda(q_2) = \{q_2\}$$

Her durumun Σ sembolleri geldiğinde nereye gidiyor?

$$\delta^*(q_0, 0) = \{q_0, q_1, q_2\}$$

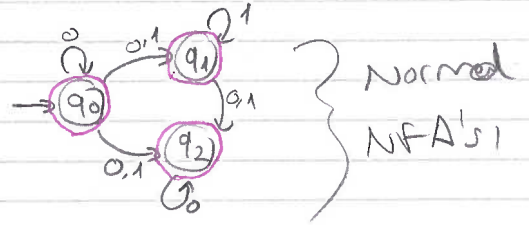
$$\delta^*(q_0, 1) = \{q_1, q_2\}$$

$$\delta^*(q_1, 0) = \{q_2\}$$

$$\delta^*(q_1, 1) = \{q_1, q_2\}$$

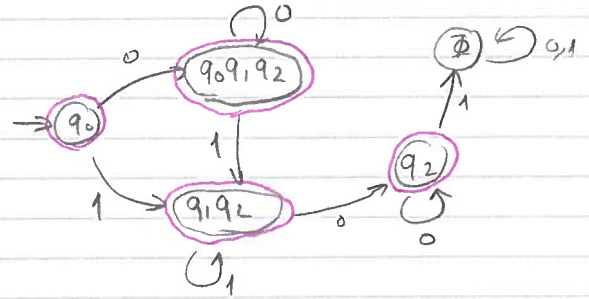
$$\delta^*(q_2, 0) = \{q_2\}$$

$$\delta^*(q_2, 1) = \{\emptyset\}$$



DFA'sini
çizelim ⇒

Q	0	1
q ₀	q ₀ , q ₁ , q ₂	q ₁ , q ₂
q ₁ , q ₂	q ₀ , q ₁ , q ₂	q ₁ , q ₂
q ₁ , q ₂	q ₂	q ₁ , q ₂
q ₂	q ₂	∅
∅	∅	∅

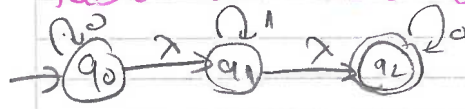


Haytays tablo kullanarak BG-NFA'yı NFA'ya dönüştürme.

Sinav

Sinavda BG-NFA ⇒ NFA ⇒ DFA dönüşümü, inşası gibi 6-7 soru içine düşer.

Toplamda 3 soru düşer, birisi 6-7 soru içine düşer.

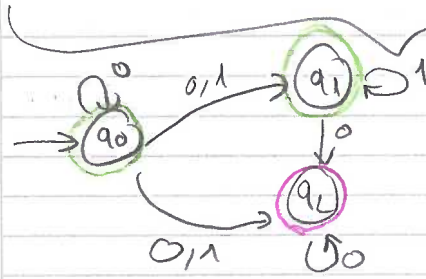
Tablo kullanarak $\text{DG-NFA} \rightarrow \text{NFA} \rightarrow \text{DFA}$ dönüşümü

Buna her zaman kendisi de dahildir.

		Bölük kapaması	asıl önemli kısım (par) burada			
		$\delta(q)$	$\delta(q,0)$	$\delta(q,1)$	$\delta^*(q,0)$	$\delta^*(q,1)$
q_0		q_1, q_2	q_0	—	q_0, q_1, q_2	q_1, q_2
q_1		q_2	—	q_1	q_2	q_1, q_2
q_2		—	q_2	—	q_2	—

* Yildizli kumda, δ^* danlara 0 gelmesi durumuna bakıyoruz. Ayrıca kendilerinin de hepüne 0 gelirler. $\delta^*(q_0,0) \rightarrow q_0$ 'ın bölük kapamasının tamamına 0 gelmesi durumu gibi hesaplanır.

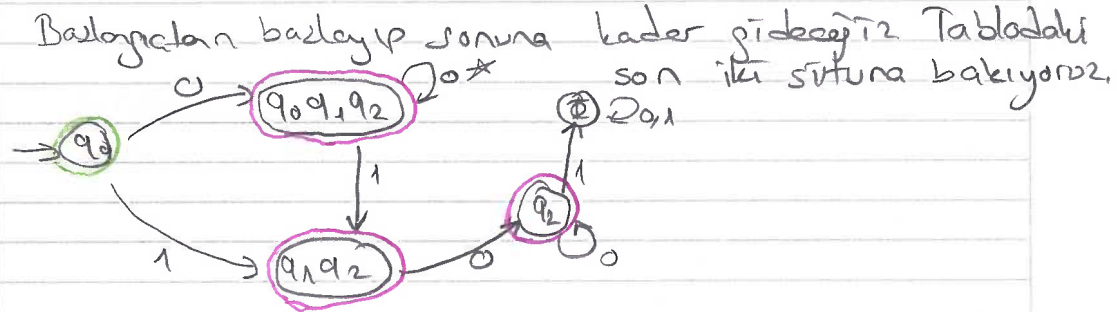
NFA çizimi



Önceki hayattaki dostteki sonuçla aynı.

Pembe kabul zaten q_2 'den geliyor. Yeşil olanlar da bölük geçişinden dolayı geliyor.

DFA çizimi

* izareti için: q_0 'a 0 gelirse, q_1 'e 0 gelirse ve q_2 'ye 0 gelirse hepsinin birleşimini aldık.

Sınav

Sınavlarda tablo veya yazma yöntemi genelde zorlanmıyor. Ama indirgenmede genelde tablo zorlanıyor.

Örnek
(Çizili değil,
yazılı örnek)

λ -NFA $M = \{Q, \Sigma, \delta, s, F\}$

$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$

$\Sigma = \{a, b\}$

$s = q_0$

$F = \{q_2\}$

$S \rightarrow (q_0, a) \rightarrow q_3$

$(q_0, \lambda) \rightarrow q_1$

$(q_1, a) \rightarrow q_4$

$(q_1, b) \rightarrow q_2$

$(q_3, b) \rightarrow q_4$

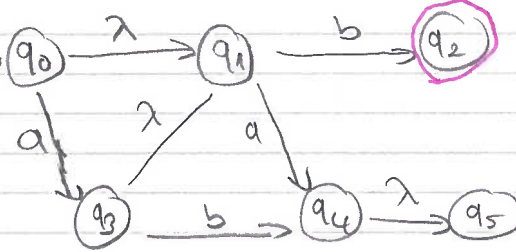
$(q_3, \lambda) \rightarrow q_1$

$(q_4, \lambda) \rightarrow q_5$

Soru: Verilen ifadenin NFA'sını ve DFA'sını çiz.

Çözüm: q_0 'den başla.

Başlık geçiri NFA



Soru: Potenti ifadesi nedir?

Çözüm: Dango yapan yer var mı diye bakarsak önce Yok.

q_0 'den başlarsak boşluklar dıyı ok gider: b/ab

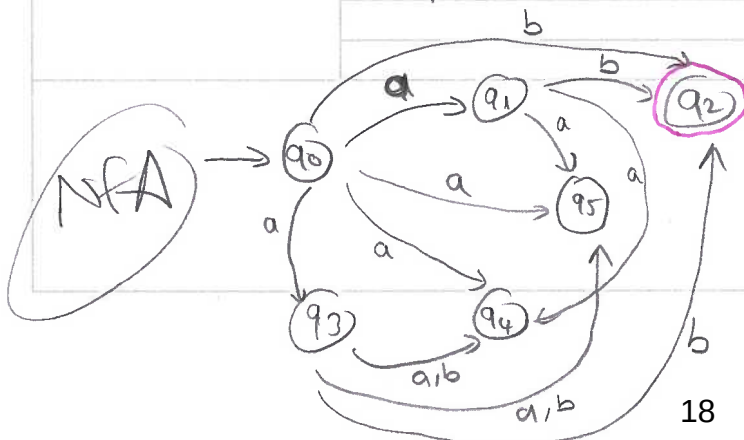
bir kabul durumuna
götüren durumlarak.

Tablo çizelim şimdi:

Son iki sütunda
bunların ikisine de
bakılmalıdır.

Q	λ	a	b	a^*	b^*
q_0	q_1	q_3	—	q_1, q_3, q_4, q_5	q_2
q_1	—	q_4	q_2	q_4, q_5	q_2
q_2	—	—	—	—	—
q_3	q_1	—	q_4	q_4, q_5	q_4, q_5, q_2
q_4	q_5	—	—	—	—
q_5	—	—	—	—	—

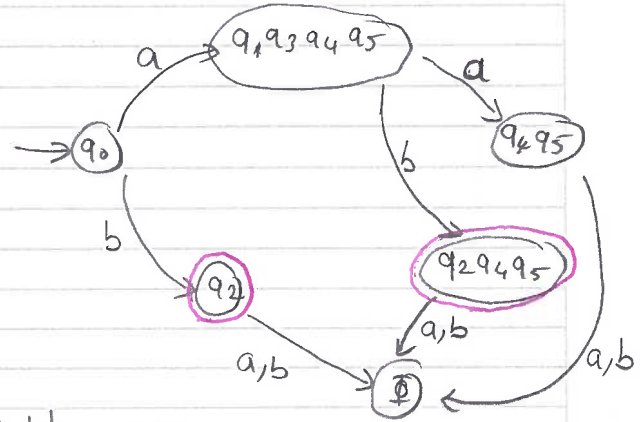
buyla da
yazılabilir.
önceki örnekteki
gibi de.



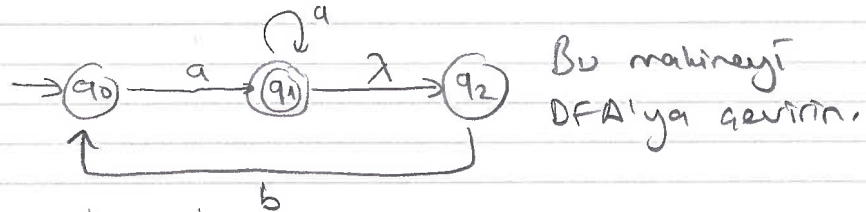
$\rightarrow q_2$ 'ye başlık geçiri ile
ulaşıldığımız yer yok. Sadece
 q_2 kalıyor.

DFA Çözümü

Q	a	b
q ₀	q ₁ q ₃ q ₄ q ₅	q ₂
q ₁ q ₃ q ₄ q ₅	q ₄ q ₅	q ₂ q ₄ q ₅
q ₂	—	—
q ₄ q ₅	—	—
q ₂ q ₄ q ₅	—	—
Φ	Φ	Φ



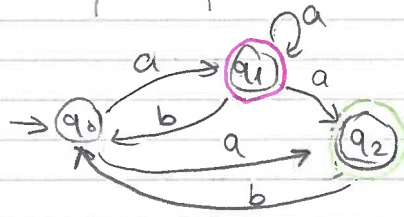
* En sade gösterim DFA'da var.
ab|b ifadesi net bir şekilde gösterilemiyor.

Örnek

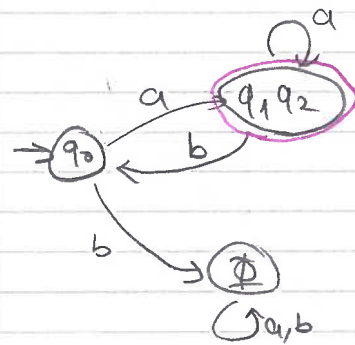
Çözüm

Q	λ	a	b	a*	b*
q ₀	—	q ₁	—	q ₁ ,q ₂	—
q ₁	q ₂	q ₁	—	q ₁ ,q ₂	q ₀
q ₂	—	—	q ₀	—	q ₀

NFA



DFA

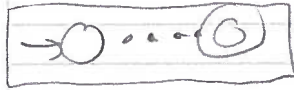
 $a^+(ab)^*$

Indagene kavusu

$L_1 \cup L_2$ ifade $L_1 + L_2$ veya $L_1 | L_2$ olarak yazılabilir.



$\Rightarrow$ en temel makine
 L_1 dilinin M_1 makinesi



$\Rightarrow L_2, M_2$

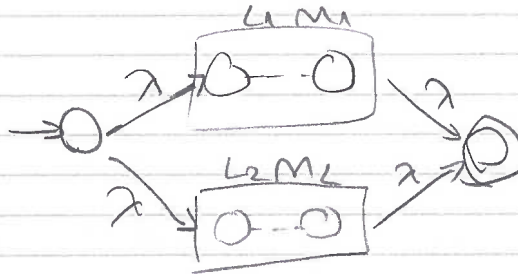
İki makineyi bitirtirmek için; 1. makinenin kabul durumunu kaldır, dışarı al. 2. makinenin de başlangıcını kaldır, dışarı durum etek. İkisinin arasına bir tane boşluk gene durumu ekler.



Bitirme
(kesim)



Birleşim
olur:



Ters: $a^n b; n \geq 0 \Rightarrow$ regül ifade eder. $\Rightarrow a^* b$ olur.

Makine: $\rightarrow \text{O} \xrightarrow{b} \text{O}$ olur.

Tersi de: ba^n olur. Regex: ba^* olur.

- Başlangıcı kaldır.
- Kabulü kaldır.
- Başlangıcı kabul yap, Kabulü başlangıç yap
- Okları ters çevir.

Regül:

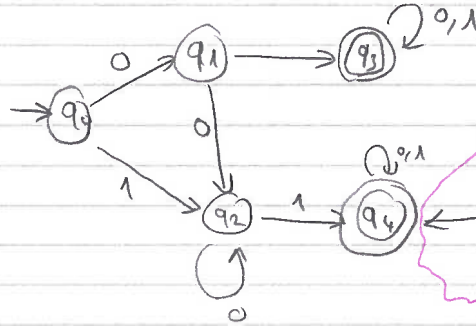
Regülde tüm durumlar belli ilgilendirir.

- Kabulü red yap
- Red'leri kabul yap

} Ancak NFA'daki belli ilgilendirmeleri gösteremediğimiz çizim olur. Tüm durumlar gösterilmelidir.

İNDİRGEN KURALLARI (2 tane)1. Aşama

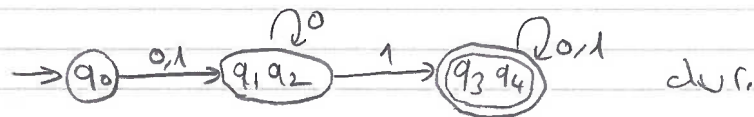
① Ulaşılamayan durumları çıkart.



1. aşamada çıkarılır, ulaşımıyor

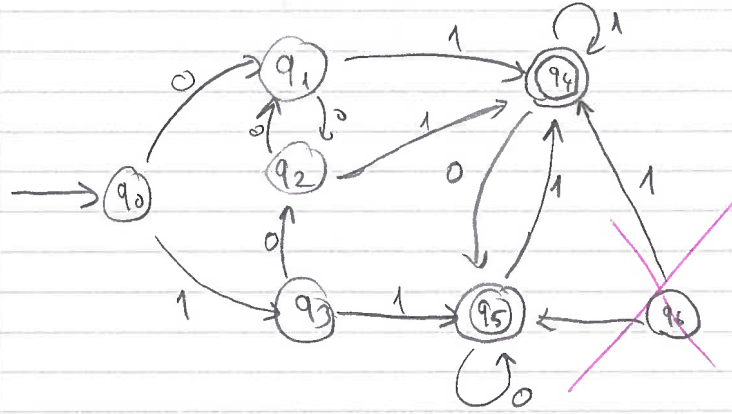
② Tablo doldurma yöntemi ile denk durumların bulunması.
- Bunun için önce değişken aranmalıdır.

q ₁	X			
q ₂	X	✓		
q ₃	X	X	X	
q ₄	X	X	X	✓
	q ₀	q ₁	q ₂	q ₃

① q₃ ve q₄ kabul olduğundan, red denli bir denk denmez. Kabul ve redler kendi içinde bakılır.2. aşamada 0 gelirse (q₁ - q₀ arasında) aynı ama 1 gelirse farklı.3. aşamada (q₂ - q₀) 1 gelirse farklı.4. aşamada ilk kez 0 gelirse (q₂ - q₁) değişken giriyor 1 gelirse yine değişken girip kabul değişken DENK5. aşamada ikisi de (q₃ - q₄) kabul DENKDenekli (q₃ - q₄) ve (q₂ - q₁) birleştirilecek.

0/1 0*1

★ İndirgen yaparken önce kabule girip girmediğine bakmak daha hızlı çözüme getirebilir.

Örnek:

Sonu indirgene yapalım.

Görüm: q6 ya ulaşamıyor, çıkartılır.

q1	x				
q2	x	✓			
q3	x	✓	✓		
q4	x	x	x	x	
q5	x	x	x	x	✓
	q0	q1	q2	q3	q4

① q4 & q5 kabul olduğundan, red olanlarla tamamını kesimini kapatıldı.

(q0-q1) → 1 gelirse birisi kabul gidiyor, diğeri gitmiyor.

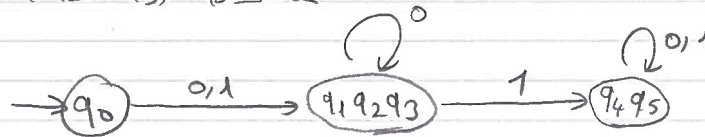
(q1-q3)	q1	q3
0	q2	q2 ✓
1	q4	q5
10	q5	q5 ✓
11	q4	q4 ✓

DENK

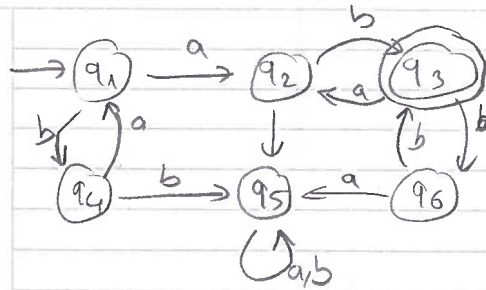
(q1-q2)	q1	q2	
0	q2	q1	
1	q4	q4 ✓	döngü
00	q1	q2 ✓	döngü
01	q4	q4 ✓	döngü

q1 ≡ q2 ve q1 ≡ q3 çıktı. Eğer q2 ≡ q3 olursa üçüde bir yerde birleşecek.

(q2-q3) DENK



Indirgenme
Örneği



(*) q_3 kabul ve diğerleri red olduğundan, hiç birisi ile denk olamaz.

(*) q_2	q_6
a q_5	q_5
b q_3	q_3

q_2	X				
q_3	X	X			
q_4	X	X	X		
q_5	X	X	X	X	
q_6	X	✓	X	X	X
	q_1	q_2	q_3	q_4	q_5

(*) q_4	q_5
a q_1	q_5
b q_5	q_5 ✓
aa	—
ab	—

* Kuayel: Her karşılaştırma için; önce birini kabule getiren kataşa bak. Sonra aynı katarın diğerini nereye götürdüğüne bak.

Pumping Lem

Bir ipaddenin DFA'sını çizilebiliyorak, regülerdir. Bir ipaddenin regüler olmadığını ispatlayabiliriz, Ancak bu onun regüler olupuna göstermez.

Örneğin bir makine olduğunu. Bunun durum sayısı " $n+1$ " olsun. Kontrol edeceğim katar sayısı da $n+1$ ise; en az bir durum tekrar etmelidir.

Eğer verilen bir dilde, içinde tekrar edilen yer varsa; bu tekrarların sayısının artırılması veya azaltılması, oradaki kabul durumunun kabullüğünü veya red ise redliğini değiştirmez.

Örnek: $L = \{a^n b^n \mid n \geq 1\}$ şeklindeki ipade regüler midir?
Cevap: Burada hiçbir kullandığı için regüler değildir.

İspat:

TEMENÜKA
TEMENÜKA

İspat: $a^k b^k$ olduğunu düşünelim. $\underbrace{aaa \dots a}_{k \text{ tane}} \underbrace{bbb \dots b}_{k \text{ tane}}$

Şimdi bu ifadeyi 3 parçaya ayıracağız: $w = x y z$

x : tekrar etmeyen kısım
 z : " " " "
 y : tekrar eden kısım } olacak.

y^i olursa ve $i \geq 0$ olursa; tekrar eden ifadelerin kabul/red durumuna değeri-
 meye bakalım.

$\underbrace{aaa \dots a}_{k \text{ tane}} \underbrace{bbb \dots b}_{k \text{ tane}}$

Regüler olmadığını ispatlayalım.

$\underbrace{aaa \dots a}_{x} \underbrace{bbb \dots b}_{y} \underbrace{}_{z}$
 değeri belli değil.

$y = a^s$, $x = a^r$, $z = a^l b^k \Rightarrow$ olur. $w = a^s a^r a^l b^k$ olur.

$y = a^0 = \lambda$ olsa $\Rightarrow w = a^r a^l b^k = a^{k-s} b^k$

baştaki ifadeden çok farklı
 bu nedenle regüler değildir.

Örnek:

$L = \{a^k b^{k+l} c^l \mid k, l \geq 0\}$ regüler midir? Sınımda
 "DFA'sını çiztin" denebiliriz. Regüler
 olmadığını ispatlamamız gerektirir.

İspat: Örneğin $a^n b^{2n} c^n$ olsun;

$\underbrace{aaa \dots a}_n \underbrace{bbb \dots b}_{2n} \underbrace{ccc \dots c}_n \Rightarrow c\text{'lerin bir kısmı } y \text{ olsun.}$

$y = c^s$ olsun. } $n-r-s$ tane C kalır. Orada p diyelim
 $z = c^r$ olsun.

y^2 alalım denemek için. $\Rightarrow y^2 = c^{2s}$ olur.

$w = x y z = a^n b^{2n} c^n$ idi.

$y = y^2$ için $\Rightarrow a^n b^{2n} c^p c^{2s} c^r$ olur.

$= a^n b^{2n} c^{n+s}$

başta seçtiğimizden farklı olupunda
 regüler değildir.

Gramer

$$G = \langle V_N, V_T, P, S \rangle$$

V_N : Non terminal kümesi $\rightarrow$ Açıklanabilir.

V_T : Terminaler kümesi $\rightarrow$ Atomik. Daha küçük parçaya ayrılmaz.

P : Yerine yazma kuralları

S : Başlangıç nonterminali

terminal $\rightarrow t$

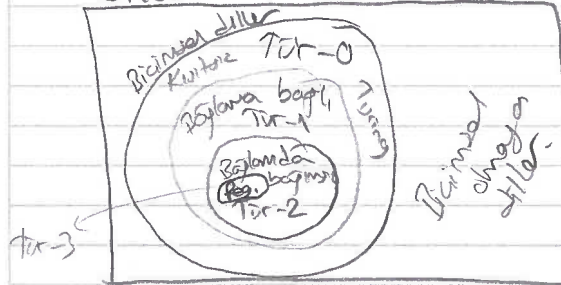
nonterminal $\rightarrow T$

$$V_1' \Rightarrow V_2'$$

Yerine yazma kuralı örneği

$$V_1' = V_T + V_N$$

$$V_2' = V_T + V_N$$

Diller

Tür-0: Turing makinesi ile tanımlı. Sınırsız sayıda terminal veya nonterminal kullanılabilir. Kısıtlı da denir. Bicimsel dillerin tümünü tanımlayabilir.

Tür-1: Bağlama bağlıdır. Soldaki ter. veya nonter.'ler, sağdakinden mutlaka azdır.

Tür-2: Bağlamda bağımsız (context-free). Solda 1 tane ter veya nonter, sağda sınırsız olabilir. Bunu tanıyanlara Push-Down Automata (PDA) denir.

Tür-3: Register diller. Bunu da finite automata'lar (DFA, NFA, λ -NFA) tanımlar. Solda 1 tane T , sağda 2 tane olur en fazla. 2 olursa $1T, 1t$ olabilir.

Bizi Tür-2 ve Tür-3 ilgilendiriyor.

Örnek 2

$L = \{a^n b^n \mid n \geq 1\}$ dilin gramerini yazın

Çözüm: Önce yerine yazma kurallarını düştürmeliyim,

$S \rightarrow aSb$ denebilir. Kural olarak. $aSb \rightarrow$ ortadan açılır.

Daha doğru $S \rightarrow a \overset{①}{S} b / \overset{②}{a} b$ olarak yazılabilir.

sınırsız tek.

Bunun terminal ve nonterminalleri yazalım!

$$G = \langle V_N, V_T, P, S \rangle = \langle N, T, P, S \rangle$$

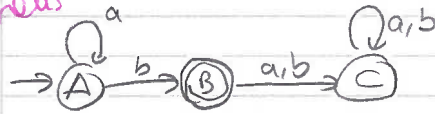
$$V_N = \{ S \} \text{ (açıklanabilen)}$$

$$V_T = \{ a, b \} \text{ (açıklanamayan)}$$

$$S = \text{başlangıç}$$

$$P \rightarrow S \Rightarrow aSb \text{ veya } S \Rightarrow ab \text{ aynı şeydir.}$$

Örnek



Tür-3 dildir. Bu.

Solken 1 tane, sağda

$$\begin{aligned} C &\rightarrow aC / bC \\ B &\rightarrow \lambda / bC / aC \\ \text{Başlangıç} &\rightarrow A \rightarrow aA / bB \end{aligned} \quad \left. \begin{array}{l} \\ \\ \end{array} \right\} \text{yerine yazma kuralı}$$

herşeyde 1t, 1T olduğundan Tür-3'tür.

Soru:

Asal sayıların DFA'sını çizim denmiş 1 kere, çizilemeyeceğini göstermeniz gerekmiştir.

Örnek $a^n(bc)^n$ ($n \geq 0$) bu dili tanımlayın. (yani yerine yazma kuralını düzütün)

$$Gösterim: S \rightarrow aSbc / \lambda \text{ dur.}$$

Örnek $L = \{ ww^R \mid w = \{a,b\}^* \}$ hangi katarları tanıtır?

$aabbbaa$ } başladığı ile bitenecek.

$$S \rightarrow \overset{①}{aSa} / \overset{②}{bSb} / \overset{③}{\lambda} \quad \left. \begin{array}{l} \\ \\ \end{array} \right\} \text{kurallar bunlardır.}$$

Yerine yazma kuralı →

Burada herhangi bir kuralı seçip, içindeki S yerine herhangi bir kuralı kayarak, katarlar türetilir.

$$\text{ör: } a \overset{①}{Sa}$$

$$\begin{aligned} &\rightarrow \text{①} \rightarrow aa \overset{①}{Sa} a \\ &\rightarrow \text{②} \rightarrow ab \overset{②}{Sb} a \end{aligned}$$

} recursive olarak yapılır.

Uyarı

- λ -NFA $\rightarrow$ NFA DFA
- bilerim, katarım, yark
- indirgenir
- pumpin lamp şimdiye kadar 1 kere çıkmış.

Örnek $L = \{a^k(bb)^s \mid k, s \geq 0\}$ tanıdığı katarlar nedir? kuralları yazalım.

Çözüm:

Hayat gerektirmez. $k \leftrightarrow s$ arasında bağlantı yok. Ter-4
örnek $a^k = A$, $(bb)^s = B$ diyelim.

- ① $S \rightarrow AB$
 - ② $A \rightarrow aA$
 - ③ $A \rightarrow \lambda$
 - ④ $B \rightarrow Bbb$
 - ⑤ $B \rightarrow \lambda$
- } Bu 5 kuralla
tüm sözcükleri tanıyabiliriz.

Soru 2: $w = aabbbb$ katarını tanıyarmı?

Çözüm: 1. kuralla başlayalım. $S \rightarrow AB$

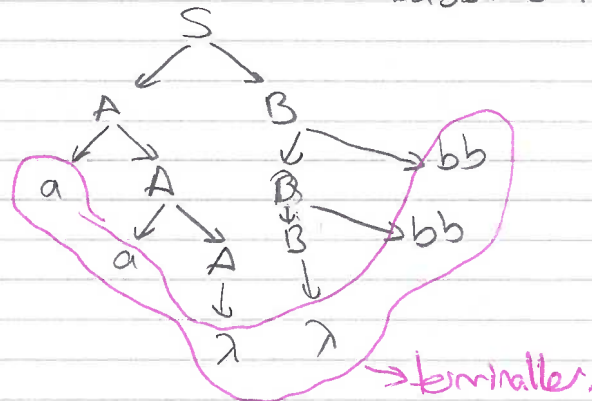
2. kurallı salıstıralım: aAB ($A \rightarrow aA$)
2. kurallı " : $aaAB$ ($A \rightarrow aA$)
3. kurallı " : aaB ($A \rightarrow \lambda$)
4. kurallı " : $aaBbb$ ($B \rightarrow Bbb$)
4. kurallı " : $aaBbbbb$ ($B \rightarrow Bbb$)
5. kurallı " : $aabbbb$ ($B \rightarrow \lambda$)

Kural sırası: 1223445 olabilir. Jinde seçilen
acmeyi deneyelim;

$S \xrightarrow{①} AB \xrightarrow{④} ABbb \xrightarrow{④} ABbbbb \xrightarrow{⑤} Abbbb \xrightarrow{②} aAbbbb \rightarrow$

$\xrightarrow{②} aaAbbbb \xrightarrow{③} aabbbb$ bu seçer 1445223 oldu.

★ Ağaca çizelim: Sonda katarların hepsi terminal rie
kabul edilir. Nonterminal kalırsa red olur.



Örnek:

Katar: begin
 $x := z;$
 $y := x + z;$
 end

Dil: program $\rightarrow$ begin dl end

dl $\rightarrow$ d | d ; dl

d $\rightarrow$ deg := ifade

ifade $\rightarrow$ deg | deg + deg $\rightarrow$ ifadesi

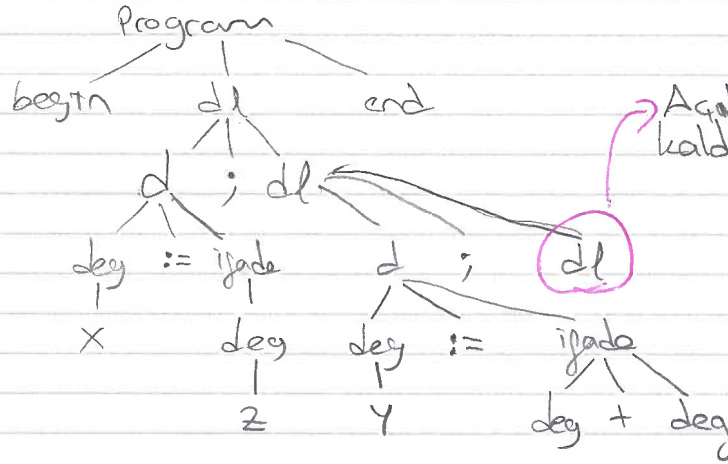
deg $\rightarrow$ x | y | z

deyim listesi

deyim

ifadesi

Verilen katar, verilen dil kuralları tarafından tanımlanır mı?



Açıklamaya nonterminal kaldığından kabul değil.

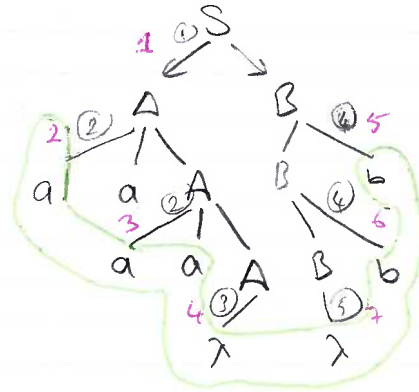
Sıra: Buradan son-
rasından paralelde cika-
cak olup, %60-
%70
arasında.

* Vire sorularının cevaplarını yaptı.

Agrostis Spae.

aaabbb

- ① $S \rightarrow AB$
- ② $A \rightarrow aaA$
- ③ $A \rightarrow \lambda$
- ④ $B \rightarrow Bb$
- ⑤ $B \rightarrow \lambda$

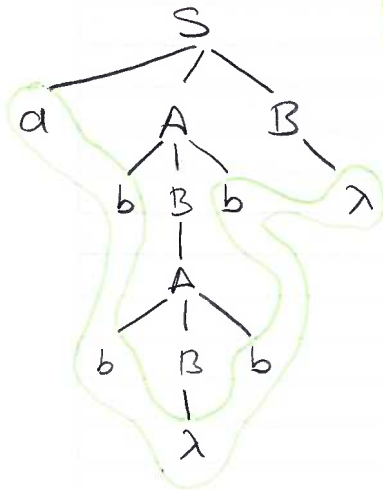
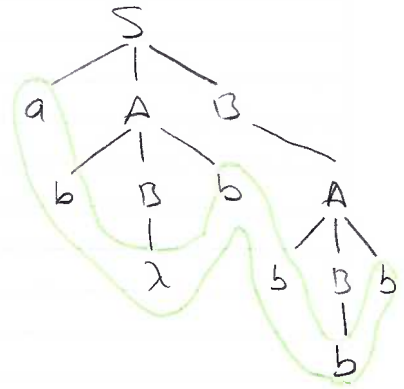
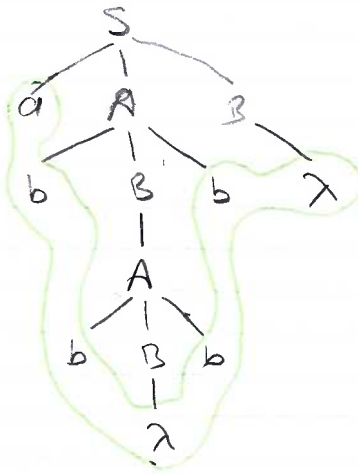

$$\begin{aligned} S &\xrightarrow{1} AB \xrightarrow{2} qaAB \xrightarrow{2} \\ &\quad qaqaAB \xrightarrow{3} qaqaqAB \xrightarrow{3} \\ &\quad qaqaqABb \xrightarrow{4} qaqaqABbb \xrightarrow{5} \\ &\quad qaqaqbb \end{aligned}$$

7 aşamada tamamen acıldı. Nonterminal kalmadı. acaalib katarı kabul edildi.

★ Sölen ve söylen acılığında aynı apas, aılmalıdır.
Aynı aılmazsa, belirsizlik dır. Belirsizlik izimize yaramaz.

Örnek: Soldan e sağdan bakalım : abbb

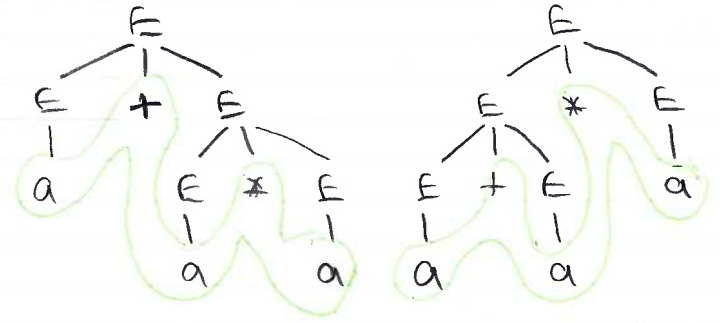
- 1 $S \rightarrow aAB$
- 2 $A \rightarrow bBb$
- 3 $B \rightarrow A$
- 4 $B \rightarrow \lambda$



Örnek:

- 1 $E \rightarrow E+E$
- 2 $E \rightarrow E * E$
- 3 $E \rightarrow (E)$
- 4 $E \rightarrow a$

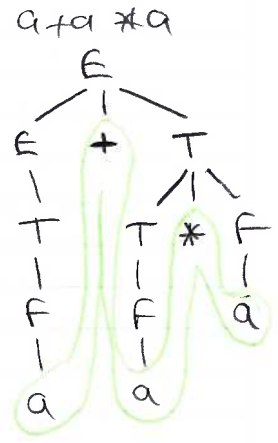
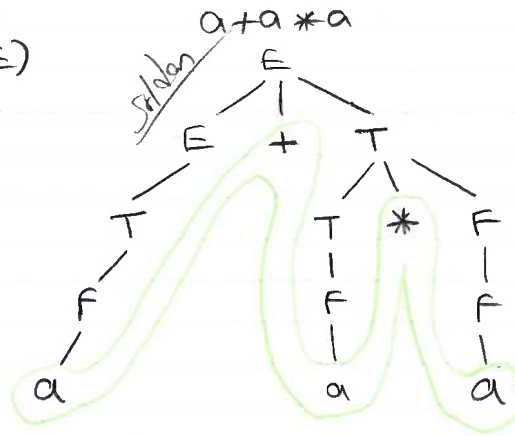
ata * a acalın (sol + sağ)



İstekte çıkan ilü apacı alttan okumaya başladığımızda mesela $a=2$ dersek; soldan açılınca 6 çıkıyor. Sağdan açılınca 8 çıkıyor. Bu durum belirsizliktir ve istenmeyen bir durumdur.

Örnek:

- Bu örnek üstteki ile aynı diti tanımlamak
- 1 $E \rightarrow E+T$ belirsizliği ortadan kaldırmak üzere
 - 2 $E \rightarrow T$ düzenlemiştir.
 - 3 $T \rightarrow T * F$
 - 4 $T \rightarrow F$
 - 5 $F \rightarrow (E)$
 - 6 $F \rightarrow a$

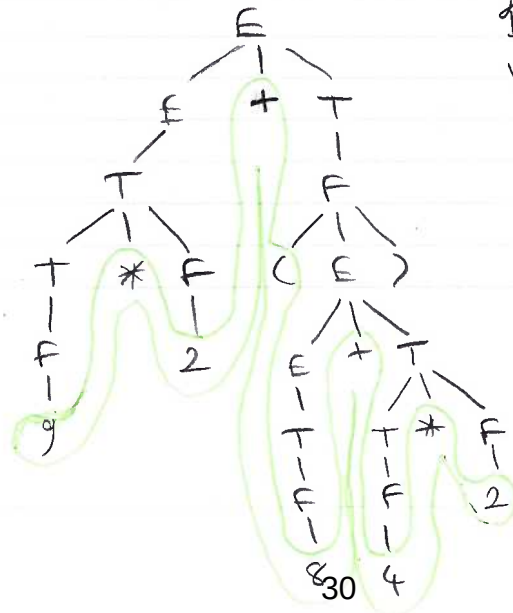


* Belirsizliği kaldırmak bizim konumuz değil.

Örnek:

$$\frac{9 * 2 + (8 + 4 * 2)}{E}$$

İstifeli örnekteki kurallar geçerlidir. a yerine "rakam" diye bir terminal tanımlı düşünüyorsak varsayıyoruz.



Chomsky Normal
formCNF

Ağaçları bu şekilde elle acmak, PC açısından verimli bir yöntem değil. Başlı listede her değişim içinde belirli sayıda yazıya işaretleri kullanmak gerekir ki bu da zor dur. Ancak buradaki yapıları hep ikili ağabilmek, PC'de bunu modellemek kolaydır. Buna Chomsky normal formu denir.

Tanım $\rightarrow V_N \rightarrow V_{N_1} V_{N_2}$ ya da $V_N \rightarrow V_T$
(ya ikiterne non terminal veya 1 tane terminal olur)

$S \rightarrow AS$
 $S \rightarrow a$
 $A \rightarrow SA$
 $A \rightarrow b$

$\left. \begin{array}{l} S \rightarrow AS \\ S \rightarrow a \\ A \rightarrow SA \\ A \rightarrow b \end{array} \right\}$ Açıklarlar non terminal
 açıklanmayanlar terminaldir.

$S \rightarrow$ ya iki tane nonterminal veya 1 tane terminalle açıklandığından Chomsky normal formu uymaktadır.

Örnek:

$S \rightarrow AS$
 $S \rightarrow AAS$
 $A \rightarrow SA$
 $A \rightarrow aa$

Chomsky'ye uygun mudur?

Cevap: $S \rightarrow AAS$ 3 tane V_N ile açıklandığından uygun değildir. Bunların CNF'ye dönüştürülmesi gerekmektedir.

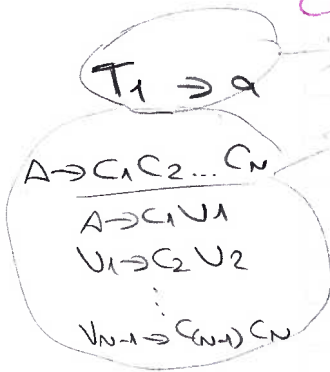
Chomsky dönüştürme adımları

- 1) her terminali bir nonterminalle gösterilebilecek hale getir. (Temsil et)
- 2) Verilen kurallarda 2'den fazla sayıda terminal/etken bir nonterminal varsa, bunlardan 1.si ayrı ayrı çıkar, geri kalanları yeni bir nonterminalle temsil edin.

Örnek: Üsttekini dönüştürelim.

$S \rightarrow AAS \Rightarrow S \rightarrow AV_1$
 $V_1 \rightarrow AS$

$A \rightarrow aa \Rightarrow A \rightarrow T_1 T_1$
 $T_1 \rightarrow a$



Örnek Chomsky örneği.

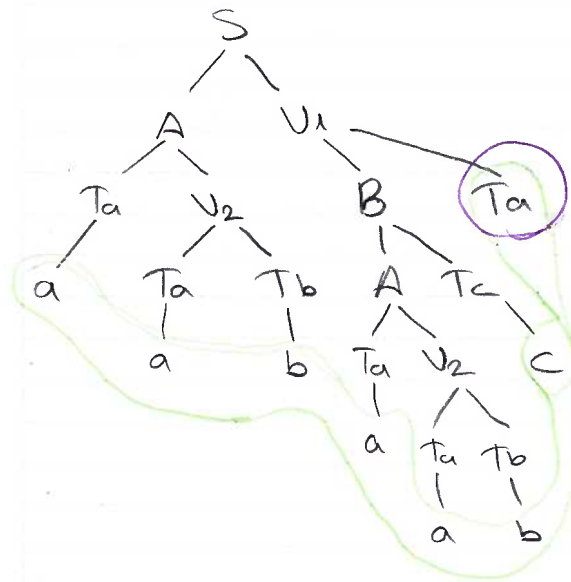
$$\begin{array}{l} S \rightarrow ABa \\ A \rightarrow aab \\ B \rightarrow Ac \end{array} \left. \vphantom{\begin{array}{l} S \rightarrow ABa \\ A \rightarrow aab \\ B \rightarrow Ac \end{array}} \right\} \text{Dönüştür bakalım:}$$

1. adım: $T_c \rightarrow c$
 $T_b \rightarrow b$
 $T_a \rightarrow a$

2. adım:

$$\begin{array}{l} B \rightarrow Ac \Rightarrow B \rightarrow AT_c \quad T_c \text{ Vn oldu, kurda uydu.} \\ S \rightarrow ABa \Rightarrow \begin{array}{l} S \rightarrow AV_1 \\ V_1 \rightarrow BT_a \end{array} \\ A \rightarrow aab \Rightarrow \begin{array}{l} A \rightarrow TaV_2 \\ V_2 \rightarrow TaT_b \end{array} \end{array} \left. \vphantom{\begin{array}{l} B \rightarrow Ac \Rightarrow B \rightarrow AT_c \\ S \rightarrow ABa \Rightarrow \begin{array}{l} S \rightarrow AV_1 \\ V_1 \rightarrow BT_a \end{array} \\ A \rightarrow aab \Rightarrow \begin{array}{l} A \rightarrow TaV_2 \\ V_2 \rightarrow TaT_b \end{array} \right\} \text{Kurallara tamamen uydu}$$

Örnek: aabaab ∈ apaca dizilim bakalım



Non terminal yaprak kaldığı için, kabul değil!

ilk dersi kaçırdım.
(Gray 5 kuru anlatım)

~~Gray~~ Gray 5 formu örneği.

$$\begin{aligned} S &\rightarrow BB|a \\ B &\rightarrow SB|b \end{aligned}$$

Görüm:

$$\begin{aligned} A_1 &\rightarrow A_2 A_2 | a \\ A_2 &\rightarrow A_1 A_2 | b \end{aligned}$$

Acaba; sağ tarafta, sol taraftaki terimin indeksinden daha düşük terim var mı? $\Rightarrow A_2 \rightarrow A_1$ var. Yerine

kayma kurallarını içleteceğiz.

$$\begin{aligned} A_1 &\rightarrow A_2 A_2 | a \\ A_2 &\rightarrow A_2 A_2 A_2 | a A_2 | b \end{aligned}$$

Doprudan recursive'lik var. 1. Adımda bunu ortadan kaldırmalıyız.

$$\begin{aligned} A_2 &\rightarrow A_2 A_2 A_2 \} A_2 \rightarrow a A_2 | b | a A_2 z | b z \\ A_2 &\rightarrow a A_2 | b \} z \rightarrow A_2 A_2 | A_2 A_2 z \end{aligned}$$

Bu ifade gray 5'e uyuyor mu? İstihale uyuyor ama alttaki uymuyor. A_2 gördüğüm her yere istihale yazalım.

$$\begin{aligned} A_2 &\rightarrow a A_2 | b | a A_2 z | b z \\ z &\rightarrow a A_2 A_2 | b A_2 | a A_2 z A_2 | b z A_2 | a A_2 z A_2 z | b z A_2 z \end{aligned}$$

A_1 halen gray 5'e uymadı.

$$A_1 \rightarrow a A_2 A_2 | b A_2 | a A_2 z A_2 | b z A_2 | a$$

şimdi, gray 5'e uydu. Her 1 tane terminal ile başlıyor. Sonra sonsuz sayıda nonterminal geliyor.

Örnek:

$$\begin{aligned}
 S &\rightarrow SS \\
 S &\rightarrow ASB \\
 S &\rightarrow AB \\
 A &\rightarrow a \\
 B &\rightarrow b
 \end{aligned}$$

Bu ifadeyi $g_{\text{ray}}S$ 'e çevirin.

1. Kural: her non terminali terminalle başlangıçla çevir.

$$\begin{aligned}
 A_1 &\rightarrow A_1 A_1 \\
 A_1 &\rightarrow A_2 A_1 A_3 \\
 A_1 &\rightarrow A_2 A_3 \\
 A_2 &\rightarrow a \\
 A_3 &\rightarrow b
 \end{aligned}$$

2. Kural: index büyüklüğü ^{sağdan} ydk. (Saplıklar, sollar $\geq$)3. Kural: derinden rekursiyi var mı?
Önce derinden rekursiyi den e dıraya diye diye ayır.

$$\begin{aligned}
 A_1 &\rightarrow A_1 A_1 \\
 A_1 &\rightarrow A_2 A_1 A_3 \mid A_2 A_3
 \end{aligned}$$

$$\begin{aligned}
 A_1 &\rightarrow A_2 A_1 A_3 \mid A_2 A_3 \mid A_3 A_1 A_3 \mid A_2 A_3 \mid \epsilon \\
 \epsilon &\rightarrow A_1 \mid A_1 \epsilon
 \end{aligned}$$

Derinden rekursiyelik kullandı.

Şimdi: Başlangıç nonterminal ise, onları terminalle çevirelim.

$$\begin{aligned}
 A_1 &\rightarrow a A_1 A_3 \mid a A_3 \mid a A_1 A_3 \epsilon \mid a A_3 \epsilon \\
 \epsilon &\rightarrow a A_1 A_3 \mid a A_3 \mid a A_1 A_3 \epsilon \mid a A_3 \epsilon \mid a A_1 A_3 \epsilon \mid a A_3 \epsilon \mid a A_1 A_3 \epsilon \mid a A_3 \epsilon
 \end{aligned}$$

zaten vardı, tekrar edilmeli.

$$\begin{aligned}
 A_2 &\rightarrow a \\
 A_3 &\rightarrow b
 \end{aligned}$$

Sonuç: tamamı $g_{\text{ray}}S$ oldu.

PDAPush Down Automata

Finite otomatlarda bellek kullanılmaz. Bunlardan ise kullanılır. PDA'da yığın bellek yapısı kullanılır.

EOF, EOL $\leftarrow$ \$ (veya z) yığının dibini, (en bastaki boş hali) gibidir.

* FA 'lar gibi 3 'lir sete benzer şekilde, burada 7 'lir set vardır. Yığının alfabesi ve yığının başlangıç durumu ilave edilmiştir 0 sete.

$$(q_1) \xrightarrow{a,b,c} (q_2) \quad w = \underset{\uparrow}{a} b b a$$

q_1 'e a geldi. Yığında b varmışçektik, yığınca c koştuk.

$$\text{SWAP} \quad \begin{bmatrix} b \\ a \\ \$ \end{bmatrix} \rightarrow \begin{bmatrix} c \\ a \\ \$ \end{bmatrix} \quad \left. \vphantom{\begin{bmatrix} b \\ a \\ \$ \end{bmatrix}} \right\} \text{Yığını değiştirmiş olduk.}$$

$$\text{PUSH} \quad \text{Yığınca ekleme de yapılabilir : } (q_1) \xrightarrow{a,x,b} (q_2)$$

$$\text{POP} \quad \text{Yığından çıkarma : } (q_1) \xrightarrow{a,b,\lambda} (q_2)$$

$$\text{Değişiklik yapılabilir : } (q_1) \xrightarrow{a,\lambda,\lambda} (q_2)$$

$$\begin{array}{c} \text{Örnek:} \\ \text{Hacın sonunda} \\ \text{verdiği örnek} \end{array} \quad \begin{array}{c} \xrightarrow{\lambda,\lambda,\lambda} (q_0) \xrightarrow{a,\lambda,a} (q_1) \xrightarrow{b,a,\lambda} (q_2) \xrightarrow{x,\$, \$} (q_3) \end{array}$$

Verilen PDA'nın tanıdığı dili yazın. Gramerini yazın.
Soru soru $\Rightarrow$ Verilen su(...) katarlar için kabul edip etmediğini yazın.

Soru: bu makine hangi grameri tanıtır? Bu PDA hangi dil türü için yazılmıştır?

Cözüm:

$$x, a, b, aa, ab, ba, bb, aaa, \dots$$

a geldikinde q_1 'de kalıyor : red

b " " q_1 'den ölü' duruma geçiyor : red

aa yine red

ab kabul

aab red. en sonunda \$ gelip \$ eklenmesini yganıyor.

aabb kabul

aabbb

PDA anlık durum yazımı

$$(q, w', y') \rightarrow q \text{ anlık durum}$$

q : bulunduğumuz durum

w' : şeritten gelen veride kontrol edilecek kalan kısım.

y' : yığındaki veri

$$w = aabb \text{ için}$$

$$(q_0, aabb, \$) \rightarrow (q_1, aabb, \$)$$

$$\rightarrow (q_1, abb, a\$)$$

$$\rightarrow (q_1, bb, aa\$)$$

$$\rightarrow (q_2, b, aaa\$)$$

$$\rightarrow (q_2, \lambda, \$)$$

$$\rightarrow (q_3, \lambda, \$) \Rightarrow \text{kabul durumunda kaldık}$$

İzin, $w = aabb$ katarı kabul eder.

Sonuç olarak: $L = \{a^n b^n : n \geq 0\}$

$$S \rightarrow ASB$$

$$A \rightarrow a$$

$$B \rightarrow b$$

$$S \rightarrow \lambda$$

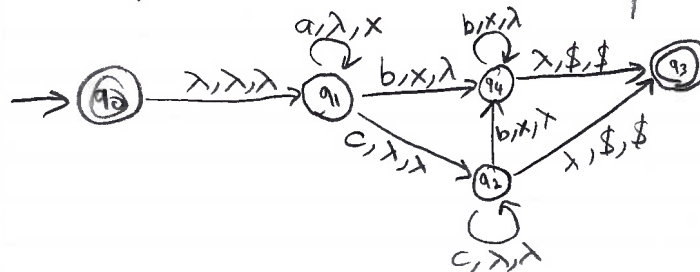
Örnek:

(Sınavda çıkmış data c'ne)

$a^n c^m b^n$ ($n, m \geq 0$) gramerini yazın, makinesini çizin.

Gramer: $S \rightarrow ASB$
 $A \rightarrow a$
 $B \rightarrow b$
 $S \rightarrow CS \Rightarrow C$ için eklendi
 $S \rightarrow \lambda \Rightarrow$ başlangıç kabul

$ccc, aabb, aacccbb$ hepsi kabul. Ağaca çizilebilir.



PDA'ya doldur-bozalt denir. Üsler yığın yapısında uygun olmalıdır.

$$a^n b^m a^n b^m \rightarrow \text{olur.}$$

$$a^n b^m a^n b^m \rightarrow \text{olmaz.}$$

$$a^n b^n c^n \rightarrow \text{olmaz.}$$

Önceden bahsedilmisti, PDA'larda 7 sembolü oluyor.

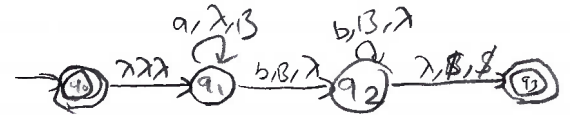
PDA makinesi: $M = \{Q, \Sigma, \Gamma, \delta, S, z, F\}$

durum $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$
 alfabe $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$
 yığın $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$
 alfabe $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$
 geçiş $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$
 başlangıç $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$
 yığın son $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$
 kabul durumu

Örnek:
 $L = \{a^n b^n : n \geq 0\}$
 $Q = \{q_0, q_1, q_2, q_3\}$
 $\Sigma = \{B\}$
 $S = q_0$
 $z = \$$
 $F = \{q_0, q_3\}$

(PDA'nın
ilk)

PDA konusundaki ilk örnek için



$w = aabb$ olsun.

$\delta(S, w, z) > (q_0, aabb, \$) > (q_1, aabb, \$) > (q_1, abb, B\$)$
 $> (q_1, bb, BB\$) > (q_2, b, B\$) > (q_2, \lambda, \$) > (q_3, \lambda, \$)$

Kabul için
1. durum $\rightarrow$
Kabul durumu

Ulaştığımız son durum, q_3 oldu. Burada posta durumu kalmadı en sonunda. Bunun F ile kesimini boş küme değilse, kabul edilir katarımız.

$\delta = (q_0, \lambda, \$) \rightarrow (q_1, \$)$
 $(q_1, a, \$) \rightarrow (q_1, B\$)$
 $(q_1, b, B) \rightarrow (q_2, \lambda)$
 $(q_1, a, B) \rightarrow (q_1, BB)$
 $(q_2, b, B) \rightarrow (q_2, \lambda)$
 $(q_2, \lambda, \$) \rightarrow (q_3, \$)$

Sıralamanın takip edemedim.

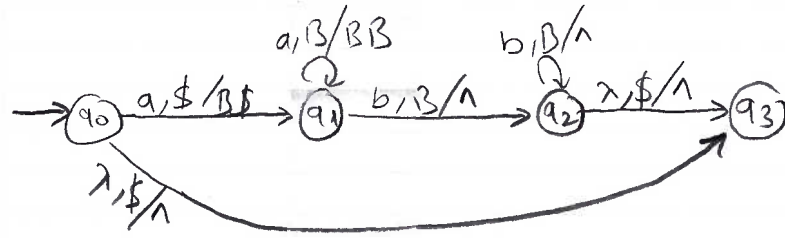
Kabul için
2. durum $\rightarrow$
(alternaty)
Empty Stack
kontrolü

$\delta(S, w, z) \vdash (q, \lambda, \lambda)$ Son geldiğimiz durumda, serinin son elemanı okunduktunda, yığında da eleman kalmamışsa, kabul edilir. Bunun için, yığının dibindeki son $\$$ da getirilmiş olmalıdır. Bu netotta, $F = \{\}$ olur.

$a, \lambda, B \rightarrow a$ geldi, büyük okuma, B ekle

Alternaty $\Rightarrow a, \$/B\$ \rightarrow a$ geldi, yığında $\$$ olmalı, yığının son $B\$$ olsun.
 swap $\rightarrow$ yığında $\$$ olsun, sonra $B\$$ olsun.

$B/\lambda \rightarrow B$ yi sil, başka birini yazma.



* Bu makine gösteriminde kabul durumları göstermez. Bunun yerine her w için, tüm geçişler yazılır ve λ ile yığın boş olması kontrol edilir.

$w = aab$ olsun.

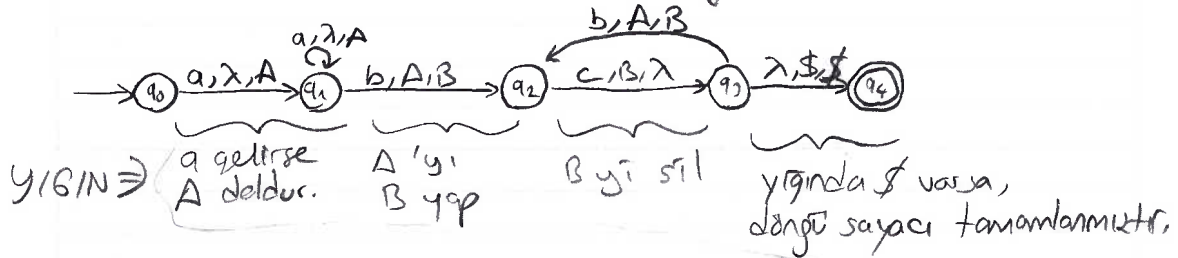
$\delta(q_0, aab, \$) \rightarrow (q_1, ab, B\$) \rightarrow (q_1, b, BB\$) \rightarrow (q_2, \lambda, B\$)$

Snayda sınırlama olmazsa, PDA örneklerinde her iki yöntemden birisi kullanılabilir.

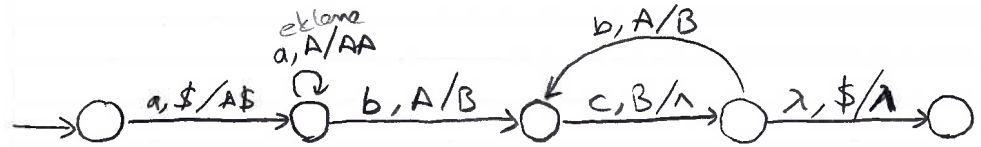
* Gerçek hayatta, yığında sembol kaldığında da kabul etmeniz gerekebilir. Bu tarz durumlarda, empty stack metodu kullanılabilir.

Örnek: $a^n(bc)^n$; $n \geq 1$ olsun.

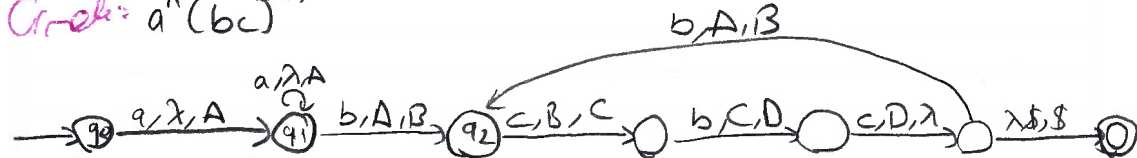
Önce kabul durumu makine yapalım.



Şimdi boş yığın kontrollü makine çizelim:

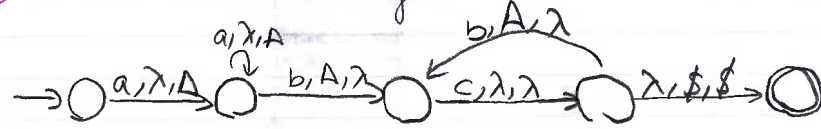


Örnek: $a^n(bc)^2n$



* Snayda PDA : 10-15 dakikada } Gözlemler.
Turing: 20-25 dakikada
Gramer de çıkar. (Ditleri yazın derler)

Örnek: $a^i (bc)^n$ için gerekli ağızım (denk)



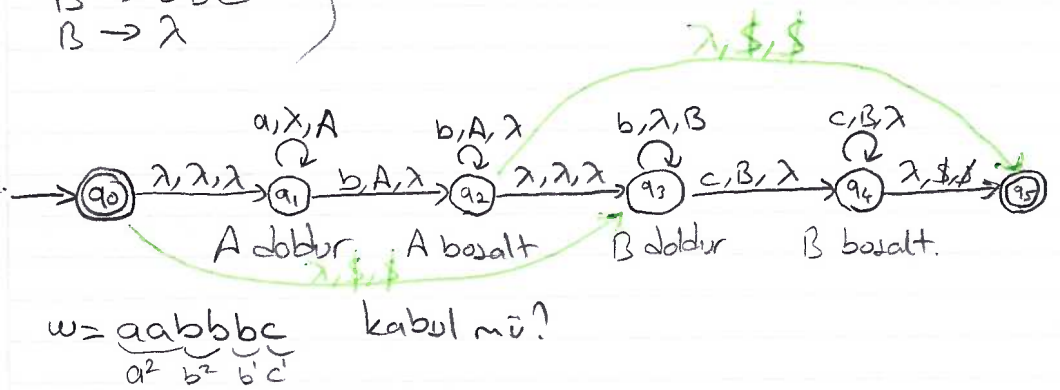
Örnek: $a^n b^{n+m} c^m$ | $n, m \geq 0$

Bunu, $a^n b^n b^m c^m$ şeklinde parçalayabiliriz.

$S \rightarrow AB$
 $A \rightarrow aAb$
 $A \rightarrow \lambda$
 $B \rightarrow bBc$
 $B \rightarrow \lambda$

Şimdi grameri sorulursa bunu yazacağız.

Yerel kısımları
sonradan ekledik.
 $n=0$ veya $m=0$
durumlarını seçer-
mek için yapıldı.



$w = \underline{a} \underline{a} \underline{b} \underline{b} \underline{b} \underline{c}$ kabul mü?
 $a^2 b^2 b^1 c^1$

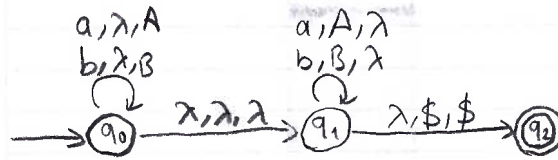
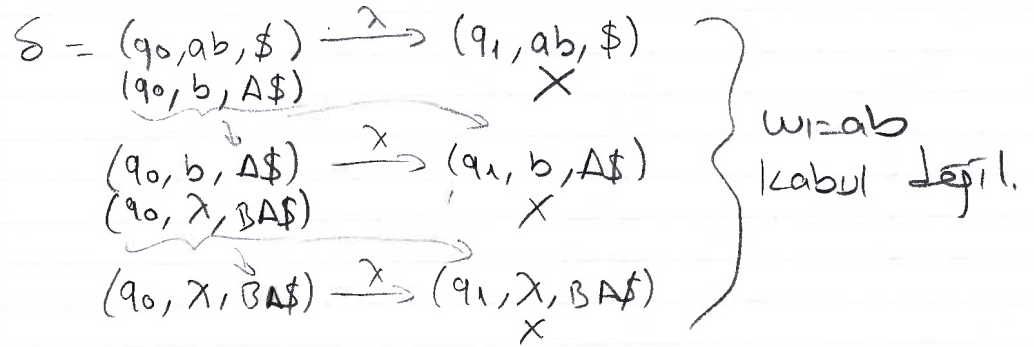
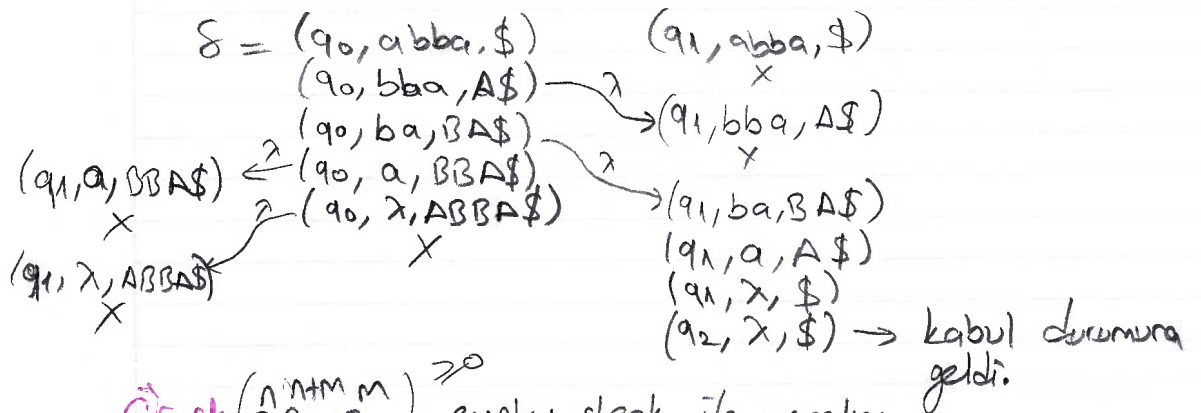
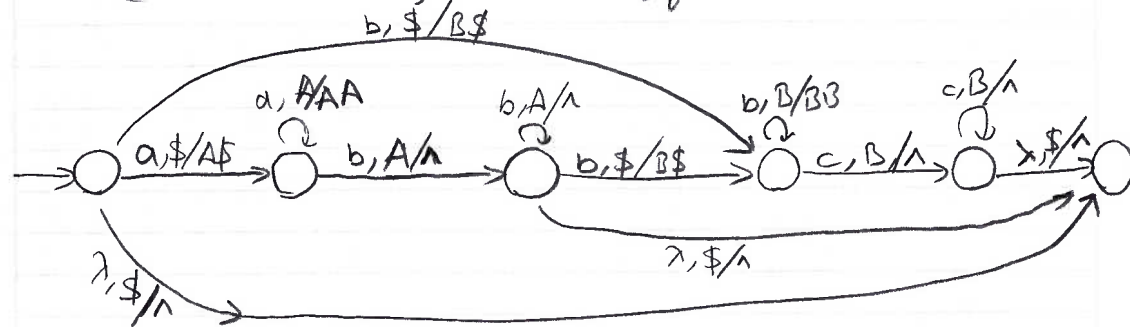
$(q_0, aabbbbc, \$) > (q_1, aabbbbc, \$)$
 $> (q_1, abbbsc, A\$)$
 $> (q_1, bbsbc, AA\$)$
 $> (q_2, bbsc, A\$) \xrightarrow{\lambda} (q_3, bbsc, A\$)$
 $> (q_2, bc, \$) \xrightarrow{\lambda} (q_3, bc, BA\$) \xrightarrow{\lambda} (q_3, bc, \$)$
 $\times$
 $(q_3, c, BB A\$) \xrightarrow{\lambda} (q_3, c, B\$)$
 $(q_4, \lambda, BA\$) \xrightarrow{\lambda} (q_4, \lambda, \$)$
 $\times$
 $(q_5, \lambda, \$)$
Kabul edilir

Örnek: VCV^R ^{reverse (tersi)}

Örnek: $a^n b^c a^p b^c a^k$ ($n, c, p, k \geq 0$)

Örnek: VV^R

Burada iki işlem yapılıyor. Yığına at ve oku.

 $w_1 = ab$  $w_2 = abba$ Örnek: $(a^n b^m c^m)^R$ empty stack ile yapalım.

Turing Makinesi

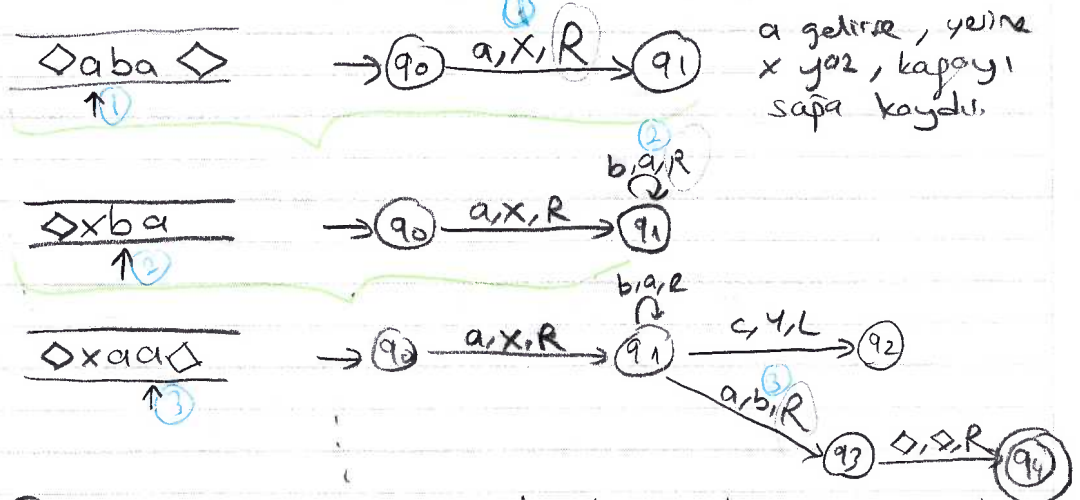
Büyük miktarlarda rasgele erişimli bellek kullanabilir. Dilin içinde karar verme yapısı olmaz. HTML'de sadece işaretler vardır. Diller ama programlama dili değildir. Yıpral soruşturma da programlama dili değildir. T-SQL veya PL-SQL ise programlama dilidir.

- * Turing makinesi; tek veya çok sırtlı olabilir.
- * Sırt; hem katarın bulundugu, hem de bellek amacı ile kullanılan alandır Turing makinesinde. Sırttaki okuma kaydı aynı zamanda yazma ünitesi de yapar.
- * Turing makinesi de PDA gibi 7'li settir.

$$\text{Turing } M = \{Q, \Sigma, \Gamma, \delta, s, \diamond, F\}$$

durumlar $\downarrow$ dil alfabesi $\downarrow$ bellek alfabesi $\downarrow$ başlangıç $\downarrow$ başlangıç $\downarrow$ belleğin hiçbirleri boş (#)

L: Left
R: Right



Durum-1

- * Turing makinesi bir zamanda kabul durumuna geldiyse, başka kontrol yapmaz, kabul edilir.
- * Sözcük bitmeden makinenin durmaması için duruma dikkat edilmelidir. Sırttaki bütün semboller mutlaka okunmalıdır.
- * Turing makinesi deterministiktir. Bir durumda iken bir kusura göre sadece 1 çıkış gösterilir. PDA'de bu konuda ayrılır. Çıkış gösterilmeyebilir veya sadece 1 tane gösterilir.

Durum-2

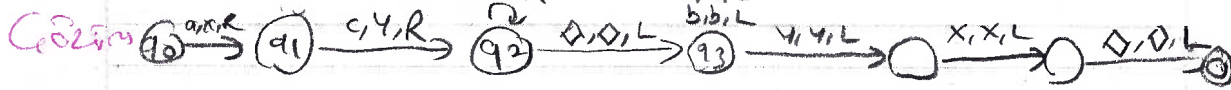
- * Turing makinesi sonsuz döngüye girerse, durur.

Durum-3
(Unexpected error occurred)

* Sırtı dururken, olmayan bir geçişle karşılaşırsa, durur ve reddeder.

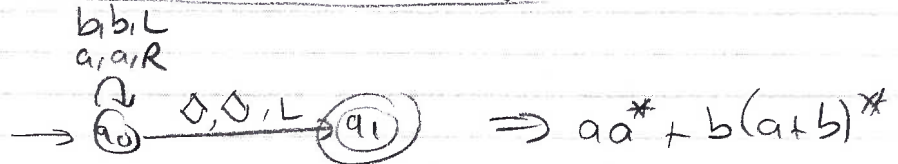
Örnek: $\langle x y a a b a b c d \rangle \quad \Sigma = \{a, b, c, d\}$

$I_{jede} = ac(a+bc)^*$ Turing ile saptayalım.



Sondaki d olmuydu, öASE sapa sonra sola gidip kabul ederdi. Bu örneğin amacı; şeridin tamamının kontrol edilmesi gereken durumlara örnek olmasıdır.

Örnek: Sonuz dögü örneği



$w = aba$ gelirse ilk iki sembol arasında sonuz dögüye girer ve katarı reddeder.

* Turing makinesi eğer DFA'yı da tanırsa o zaman regüler ifadeyi yazılır. Mesela $a^n b^n$ 'in Turing makinesi çizilebilir ama regüler ifadeyi yazamaz. Bu Turing makinesi regüler değildir.

Bosluk yok! * Turing makinesi boşluk kabul etmez.

Örnek: $a^n b^n$ Turing ile çizelim. $L = \{a^n b^n \mid n \geq 1\}$

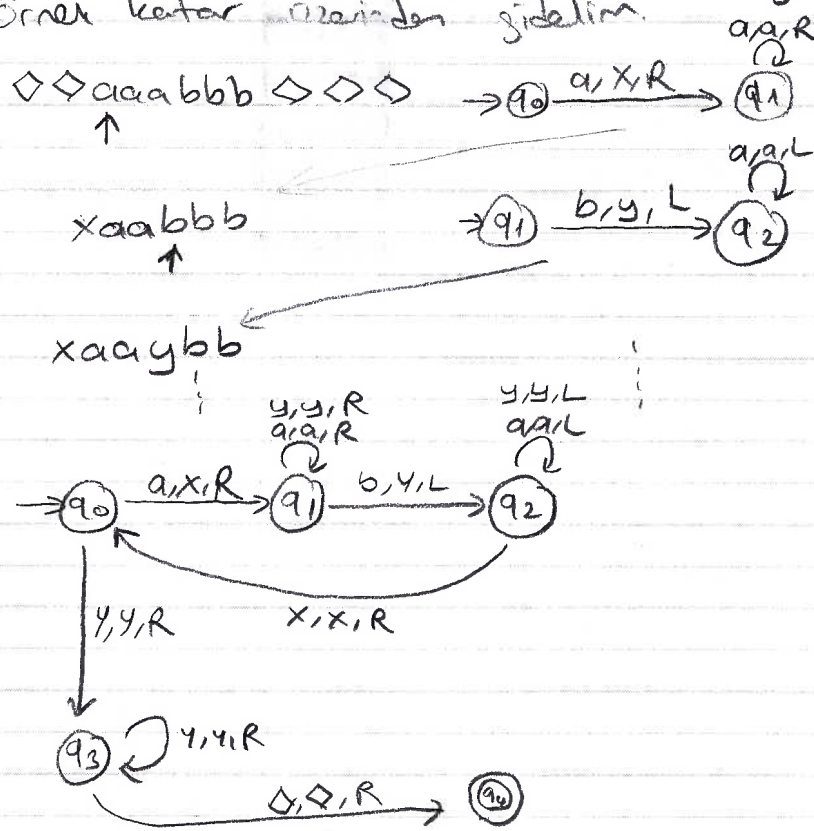
Çözüm: Mesela $aaabbbb$ kabul etmelidir.



1. yöntem: ilk a 'yı gör. Buna karşılık son b 'yi bul. Sonra bunu içe doğru recursive devam et.

2. yöntem: ilk a 'yı x ile değiştir. Buna karşılık ilk b 'yi y yap. Sonra bura dan, x 'ten sonra a varsa onu x yap. Sonra ilk b 'yi y yap. Devam et.

Makinesini çizelim. Sıradaki da böyle yapılmalıdır.
Örnek katar üzerinde gidelim.



Örnek: Üstteki makine için.
 $w = a a a b b b$ kabul mü değil mi gösterelim

$$Q = \{q_0, q_1, q_2, q_3, q_4\}$$

$$\Sigma = \{a, b\}$$

$$\Gamma = \{x, y\}$$

$$S = q_0$$

$$F = q_4$$

$$\delta \rightarrow (q_0, a) \rightarrow (q_1, x, R)$$

$$(q_1, a) \rightarrow (q_1, a, R)$$

Geciler için ikinci yöntem söyleriz:

SW

$q_0 = a a a b b b > x q_1 a a b b b > x a q_1 a b b b >$
 $x a a q_1 b b b > x a q_2 a y b b > x q_2 a a y b b >$
 $q_2 x a a y b b > x q_0 a a y b b > x x q_1 a y b b >$
 $x x a q_1 y b b > x x a y q_1 b b > x x a q_2 y y b >$
 $x x q_2 a y y b > x q_2 x a y y b > x x q_0 a y y b >$
 $x x x q_1 y y b > x x x y q_1 y b > x x x y y q_1 b >$
 $x x x y q_2 y y > x x x q_2 y y y > x x q_2 x y y y >$

$x x x q_0 y y y > x x x y q_3 y y > x x x y y q_3 y >$

$x x x y y y q_3 \diamond > x x x y y y \diamond q_4 \diamond > \text{Kabul}$

* yıldız
kapanma
gösterimi
* ile de
gösterili

Örnek: $w = abb$ kabul eder?

Çözüm:

$q_0abb > x q_1bb > q_2xyb > x q_0yb > xy q_3b$

q_3 'e b gelme durumu makinede yok. Bu nedenle red edilir.

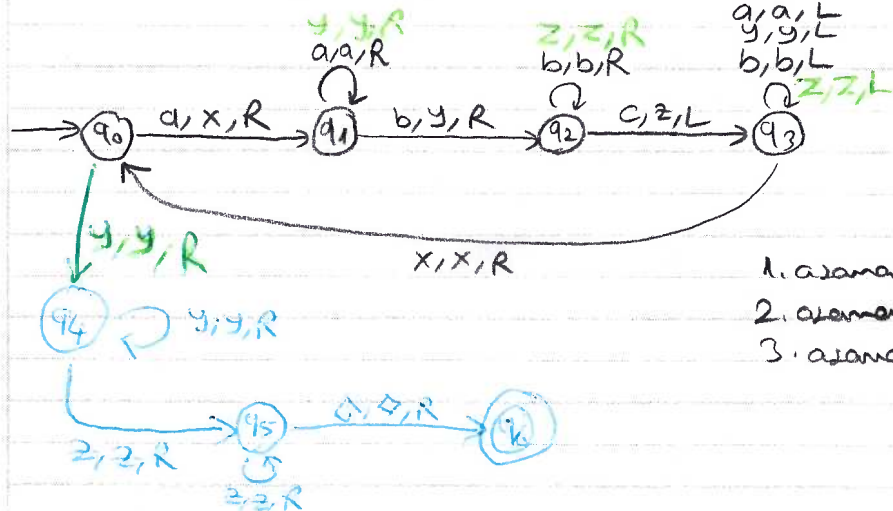
Örnek:

$L = \{a^n b^n c^n \mid n \geq 1\}$ Turing cizelim.

Çözüm: PDA ile bunu gözemiyorduk. Çünkü yipinde bir dur bozalt var. $a^n \rightarrow ddd$ dur. $b^n \rightarrow bbb$ dur. $c^n \rightarrow ?$ kaldı. Turing'te bu örneğin çözümü için 2 tane yöntem var.

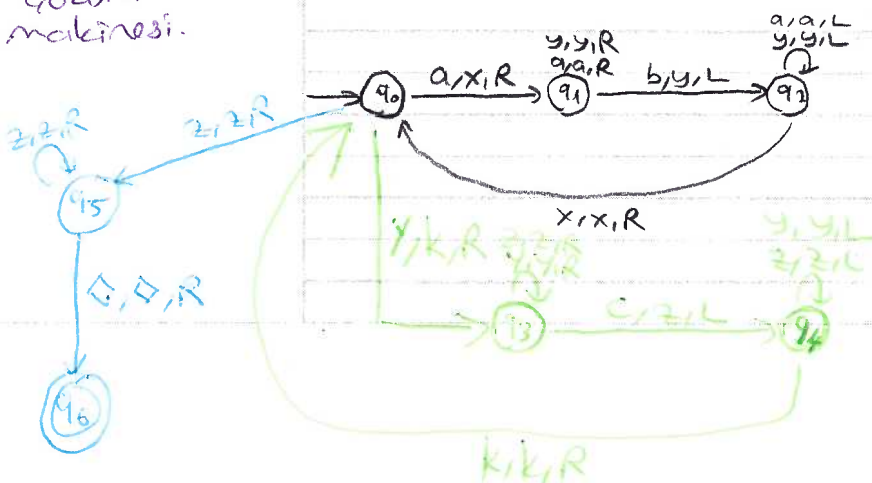
$aabbcc$ biçiminde olursa, her bir a için bir tane b ve bir tane c bulmayı lazım.

1. yöntem
çözüm
makinesi.

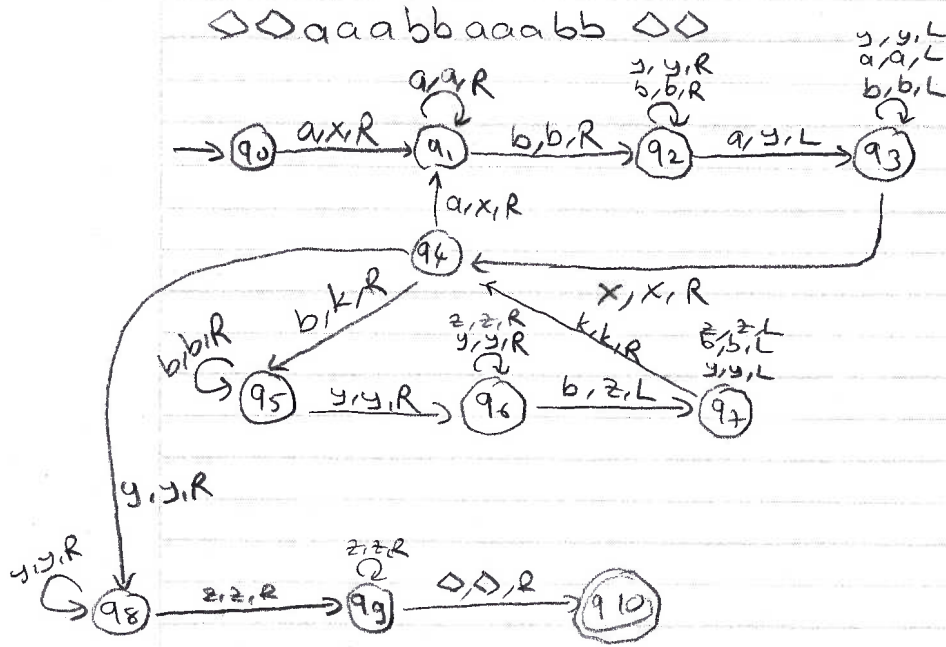


2. yöntem
çözüm
makinesi.

2. yöntemde, önce $a^n b^n$ kısmını çözelim. Sonra b 'lere karşılık c 'leri kontrol edelim.



$$\text{örnek: } L = \{a^n b^m a^n b^m \mid n, m \geq 1\}$$



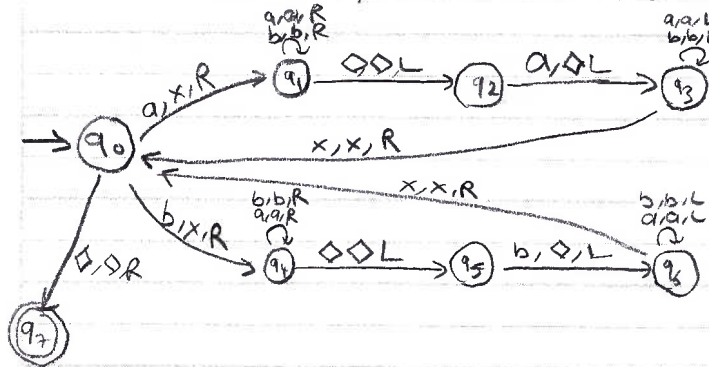
Sırası dereyelim: aabbaabb , aabab , aabbaaab
 tamamlı tamamlı tamamlı

<http://turingmachine.simulator.com>

$$\text{örnek: } L = \{ww^R\} \quad \Sigma = \{a, b\}$$

$w = abaaabab \rightarrow \text{örnek}$

Çözüm: İsterken işaret koyarak git; isterken bir baktın, bir soldan sembol sil.



Örnek: $L = \{a^n b^m c^n \mid n \geq m\}$ gövne:

Görüm: Burada $n-m$ bağı olduğu için, PDA ile çözülmez Turing ile çözelim. $n, m \geq 0$ olsaydı PDA dır.
Örnek sözde $aaabbbccc$ olabilir.

Turing ile Hesp Yaptırması (Sınavda dahil değil)

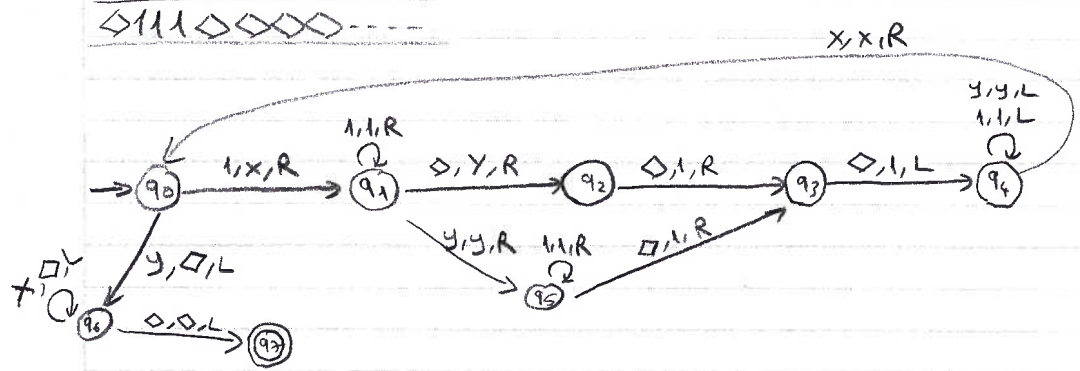
$\Sigma = \{1, 3\}$

$f(n) = 2n$ olsun. $n \rightarrow$ katardeki 1 sayısı olsun.

◇◇ 11 --- ◇◇
n tane

katarde 1 varsa onun 2 kati 1
olun sonucunda.

Görüm



1. adım: x 11 y 11 ◇

2. adım: x x 1 y 11 11 ◇

3. adım: x x x y 11 11 11 ◇

4. adım: ◇ ◇ ◇ ◇ 11 M M ◇ ◇ ..